

결정론적 가드레일

확률적 AI 에이전트를 형식 검증으로 묶다 — Lean-Agent Protocol 정독과 한국
IT 각색

· 위기 → 계보 → Lean 4 → Aristotle → 승인 루프 → 규제·보안 → 시장
· 전자정부·CMS/PMS·Slack·웹디자인·Flutter·React Native·TypeScript·Rust·Go·Python 각색
· arXiv 2604.01483 정독 + 관련 논문 10편 한국어 각색

원문 Devakh Rashie·Veda Rashi(Thomas Jefferson HSST) · 한
국어 각색 KLIC 지음

차례

들어가며	3
01 위기의 정체	6
02 가드레일의 세 계보	9
03 Lean 4의 문법	12
04 Aristotle과 자동 형식화	15
05 결정론적 승인 루프	18
06 규제 매핑 — 한국 금융으로	21
07 설명할 권리와 역방향 형식화	24
08 공격 표면과 WASM 샌드박스	27
09 시장과 선례	30
10 전자정부프레임워크의 가드레일	33
11 CMS·PMS·Slack의 가드레일	36
12 UI를 짓는 언어들의 가드레일 — Flutter·React Native·TypeScript	40
13 시스템을 짓는 언어들의 가드레일 — Rust·Go·Python	44
14 로드맵과 결론	48

DAY

들어가며



“원문: Type-Checked Compliance: Deterministic Guardrails for Agentic Financial Systems Using Lean 4 Theorem Proving — arXiv 2604.01483(2026) · Devakh Rashie-Veda Rashi(Thomas Jefferson High School for Science and Technology)”

“링크: <https://arxiv.org/html/2604.01483v1>”

“코드: <https://github.com/arkanemystic/lean-agent-protocol>”

확률의 시대에 확실성을 묻다

인공지능은 ‘답을 해 주는’ 단계를 지나 ‘일을 하는’ 단계로 넘어갔다. 목적을 이해하고, 계획을 세우고, 도구를 호출해 실제 세계의 API를 움직이는 에이전트가 그 주인공이다. 그런데 여기에 구조적인 긴장이 숨어 있다. 에이전트의 두뇌인 대형 언어 모델(LLM)은 본질적으로 확률 기계다. 다음에 올 말을 통계적으로 가장 그럴듯하게 고를 뿐, 수학적으로 확실한 답을 내놓지 않는다. 반면 돈과 규제가 움직이는 세계 — 특히 금융 — 는 확실성을 요구한다. “99.9퍼센트는 막았다”는 말이 통하지 않는 곳이다. 0.1퍼센트의 탈출이 법 위반이고, 때로는 시장 전체의 불안이 된다.

이 책이 정독하는 논문은 이 긴장에 대한 과감한 답을 내놓는다. 자연어로 쓰인 기관의 규정을 Lean 4라는 형식 증명 언어로 번역해 두고, AI 에이전트가 하려는 모든 행동을 수학적 추측으로 취급한다. 그리고 증명이 되면 실행을 허가하고, 증명이 안 되면 무조건 막는다. 확률이 남은 자리에 수학을 세우는 것이다. 이 문지기를 논문은 Lean-Agent Protocol이라 부른다.

놀라운 것은 이런 엄격함이 느리지 않다는 점이다. 아마존웹서비스(AWS)는 접근 제어 언어 Cedar를 Lean 4로 형식 검증하면서, 입력 하나를 검사하는 데 평균 5마이크로초밖에 걸리지 않는다는 것을 실측했다. 잘 튜닝된 러스트(Rust) 코드보다 빠르다. ‘형식 검증은 무겁다’는 상식을 뒤집는 숫자다.

이 책의 구성

책은 세 개의 고개로 이루어져 있다.

1부(1장~9장)는 원문 논문의 정독이다. 왜 확률적 에이전트와 결정론적 규제가 충돌하는지(1장), 지금 시장의 가드레일이 왜 부족한지(2장), Lean 4와 형식 증명이라는 낯선 도구를 강의하듯 풀어 놓았다(3장). 이어 논문의 심장부를 차례로 지낸다. 자연어 규정을 형식 코드로 옮기는 Aristotle 모델(4장), 모든 행동을 증명으로 걸러내는 승인 루프(5장), 미국 금융 규제와의 매핑(6장), ‘설명할 권리’와 감사 추적(7장), 공격 표면과 샌드박스(8장), 그리고 시장의 경쟁 구도와 실제 선례(9장).

2부(10장~13장)는 한국의 IT 지형으로 옮긴 각색이다. 원문의 통찰을 네 개의 익숙한 영역에 대입해 본다. 행정 시스템과 전자정부프레임워크(10장), 콘텐츠 관리와 협업 도구 — CMS·PMS·Slack과 웹디자인(11장), UI를 짓는 언어들 — Flutter·React Native·TypeScript(12장), 시스템을 짓는 언어들 — Rust·Go·Python(13장). 각 장은 해당 분야의 실제 논문을 함께 읽으면서, ‘증명의 가드레일’이라는 개념이 우리의 일상 도구에서 이미 어떤 축소판으로 존재하는지 보여 준다.

세 번째 고개(14장)에서는 원문의 3단계 구현 로드맵을 정리하고, 2부의 각색 논의를 다시 끌어와 한국의 개발자와 기획자가 어디서부터 시작할 수 있을지를 묻는다.

각색에 대하여

분명히 둘 것이다. 10장부터 13장까지의 비유와 적용은 원문 저자의 주장이 아니다. 원문은 금융 규제와 Lean 4를 다룰 뿐, 전자정부프레임워크도 디자인 토큰도 이야기하지 않는다. 그 영역들은 이 책을 읽으면서 필자가 원문의 개념 — 정책

환경, 증명, 승인 루프, 샌드박스 — 를 우리 현장에 옮겨 본 것이다. 각 장의 논거가 되는 추가 논문들은 각 장 머리에 출처를 밝혀 두었다. 원문의 수치와 사실은 그대로 옮겼고, 필자의 해석이 개입하는 지점에서는 본문에서 그렇게 밝힌다.

네 개의 용어

2부를 읽을 때 다음 네 용어가 기준점이 된다. **정책 환경**은 기관의 규칙을 Lean 4 코드로 번역해 둔 불변 저장소다. 규칙이 한번 컴파일을 통과하면 그 누구도, 심지어 에이전트도 임의로 고칠 수 없다. **추측**은 에이전트가 하려는 행동 하나하나를 수학 명제로 바꾼 것이다. “이 주문은 한도 안에 있다”라는 주장이 아니라 “이 주문이 한도 안에 있음이 증명 가능하다”로 옮긴 형태다. **증명과 반증**은 그 명제에 대한 커널의 이행 판정이다. 증명되면 게이트가 열리고 실행이 흘러가고, 반증되면 오류 흔적과 함께 차단된다. 확률의 여지가 없는 이분법이 핵심이다. 마지막으로 **샌드박스**는 증명을 통과해 나온 코드마저 갇아 두는 실행 울타리다. 이 책은 주로 WASM을 그 재료로 말하지만, 개념적으로는 샌드박스가 곧 마지막 보루다.

이 네 단어만 쥐고 있으면 어느 도메인의 각색 장을 읽어도 길을 잃지 않는다. 행정의 결재도, 콘텐츠의 발행 승인도, 타입 검사의 통과 판정도 결국 같은 문형의 변주다. 무엇이 불변의 규칙이고(정책 환경), 무엇이 심사받아야 하는 행동이며(추측), 누가 이행·불이행을 가르는가(증명과 반증), 통과했더라도 무엇이 남은 위험인가(샌드박스). 네 질문을 도메인의 언어로 다시 쓰면 그것이 곧 각색이다.

읽는 법

형식 증명에 대한 사전 지식은 필요 없다. Lean 4가 처음인 독자는 3장에서 개념의 뼈대를 얻고, 4장과 5장에서 그것이 실제로 돌아가는 모습을 본다. 바쁜 독자는 1장과 5장, 그리고 관심 있는 각색 장(10~13장)만 읽어도 논문의 청사진은 잡힌다. 각 장은 되도록 자립적으로 썼다. 다만 2부는 1부의 네 가지 용어 — 정책 환경, 추측, 증명과 반증, 샌드박스 — 를 알고 있으면 훨씬 수월하다.

코드를 직접 다루는 독자에게는 12장과 13장이 낯설지 않을 것이다. 우리는 이미 매일 컴파일러와 린터와 타입 시스템이라는 이름의 가드레일 속에서 일하고 있다. 이 책의 결론은 그 계단을 끝까지 오르면 어디에 닿는지를 보여 주는 것이다. 타입 검사가 축소된 증명이라면, 증명은 완성된 타입 검사다. 그리고 이제 그 완성형이 확률적 에이전트의 시대를 여는 열쇠가 되었다.

DAY

01

위기의 정체



2012년 8월 1일, 뉴욕증권거래소의 개장 직후 한 회사의 알고리즘이 통제를 잃었다. 45분 동안 4억 4천만 달러가 사라졌다. 회사 이름은 나이트 캐피털이고, 이 사건이 미국 규제 지형을 어떻게 바꿨는지는 6장과 9장에서 자세히 다룬다. 지금 주목할 것은 사건의 성격이다. 사람이 잘못 누른 것이 아니다. 배포 과정에서 되살아난 오래된 코드가 스스로 주문을 쏟아냈고, 그것을 멈출 문이 시스템 안에 없었다. 기계는 판단하도록 만들어지지 않았지만, 멈추도록도 만들어지지 않았다. 금융 시장은 그 빈자리의 값을 그대로 치렀다.

십수 년이 지난 지금, 같은 빈자리가 더 큰 문제로 돌아왔다. 이번에 입구에 서는 것은 확률 기계다.

예측하는 기계에서 실행하는 기계로

인공지능의 자리가 바뀌었다. 한때 인공지능은 과거 데이터를 읽고 미래를 추측하는 예측 분석 도구였고, 이어 사람이 확인하는 글을 대신 써 주는 생성 텍스트 모델이었다. 두 단계의 인공지능은 답을 내놓았지만 실행은 사람이 했다. 지금은 그 마지막 단계도 넘겼다. 완전한 자율 에이전트의 시대다.

에이전트는 사용자의 복잡한 의도를 스스로 해석한다. 흩어진 맥락 데이터를 동적으로 종합해 여러 단계의 실행 계획을 세운다. 계획의 각 단계는 도구 호출이고, 그 호출의 결과가 다시 다음 단계의 조건이 된다. 그리고 애플리케이션 프로그래밍 인터페이스(API)와 도구 호출 메커니즘을 통해 외부 환경과 직접 상호작용한다. 주문을 넣고, 계좌를 조회하고, 시스템을 움직인다. 물어보는 존재에서 손을 쓰는 존재로 넘어온 것이다.

금융 서비스에서 이 전환의 이득은 크다. 알고리즘 트레이딩, 동적 리스크 평가, 자율 포트폴리오 관리 같은 영역에서 운용 효율이 눈에 띄게 올라간다. 사람이 밤에도 돌아가는 시장을 상대하는 부담이 줄어들고, 의사결정의 속도가 데이터의 속도를 따라잡는다.

그러나 이득의 그림자 뒤에 구조적 위기가 서 있다. 논문은 이를 실존적 아키텍처 위기라고 부른다. 표현의 강도는 논문의 것이지만, 그 강도를 뒷받침하는 논증은 이어지는 장들에서 차곡차곡 쌓인다. 실행의 주체가 바뀌면 책임의 구조도 바뀌어야 하는데, 기존 아키텍처에는 그 바뀐 책임을 받아낸 자리가 없다. 위기는 두 얼굴로 나타난다. 한 얼굴은 시스템 신뢰성이다. 확률 기계가 돈을 움직이는 시스템은 언제 어디서 무너질지 시스템 스스로 말해 주지 못한다. 다른 얼굴은 규제 준수다. 행동 하나하나가 법적 의무의 이행이 되는데, 그 이행의 근거를 사후에 대답할 수 있어야 한다. 이 두 질문 앞에서 지금까지의 소프트웨어 관행이 더는 통하지 않게 된 것이다.

멈추는 기계와 그럴듯한 기계

전통적인 기업 자동화는 결정론적 시스템 위에 서 있다. 로봇 프로세스 자동화(RPA) 봇과 하드코딩된 워크플로우 엔진이 그 대표다. 이들은 정의해 둔 논리 경로만 수학적 정밀성으로 따라간다. 접한 조건이 정의돼 있지 않으면 실행이 멈춘다. 이 멈춤이 바로 운영 리스크의 경계선이다. 미정의 상황에서 시스템이 스스로 손을 떼기 때문에 피해가 퍼질 시간조차 주지 않는다. 경직이 곧 안전인 구조다.

LLM은 정반대로 움직인다. 고차원의 연속 벡터 공간을 항해하며 출력을 만드는데, 그 출력은 수학적으로 확실한 것이 아니라 통계적으로 그럴듯한 것이다. 같은 질문에 다른 답이 나올 수 있고, 그 답이 틀렸는지는 내부에서 알려 주지 않는다. 이 확률적 성질 덕분에 모호한 입력에도 잘 적응한다. 정의하지 못한 상황 앞에서 멈추지 않고 대응을 시도한다. 대가도 분명하다. 환각이 발생한다. 대화가 길어지면 컨텍스트 드리프트로 방향을 잃는다. 적대적 프롬프트 인젝션에 노출되면 의도가 탈취된다. 세 가지 모두 확률 기계의 체질에서 오는 병이다.

두 구조의 차이를 한 줄로 압축하면 이렇다. 워크플로우 엔진은 모르는 상황에서 멈춘다. LLM은 모르는 상황에서도 그럴듯한 답을 지어 낸다. 자동화의 세계에서 후자가 더 위험한 성질이다.

확률이 통과할 수 없는 무대

이 병이 얼마나 치명적인지는 도메인이 정한다. 글로벌 금융시장은 그 치명타가 가장 큰 곳이다. 알고리즘이 단 한 번 벗어나도 파국적 자본 고갈이나 시장 전체의 불안정으로 직결될 수 있다. 다른 산업에서 값싼 실패가 금융에서는 값지다. 주문

은 즉시 체결되고, 체결된 주문은 취소할 수 없으며, 피해는 연결된 모든 시장으로 번진다. 2012년의 그 45분이 이 경로의 실물이다. 정의되지 않은 조건 하나가 살아난 것만으로 자본이 타들어 갔다. 엄격한 제약 없는 확률적 실행은 이 무대에서 아키텍처적으로도 법적으로도 유지 불가능하다.

여기서 감각을 바로잡을 필요가 있다. 공학에서는 오류율이 품질의 척도다. 99.9퍼센트의 정확도면 훌륭한 시스템이다. 규제 세계에서 오류율은 품질의 척도가 아니라 위반의 크기다. “대부분 막았다”는 방어는 성립하지 않는다. 통과 한 번이면 충분하기 때문이다. 규제와 시장이 요구하는 것은 빈도가 아니라 보장이다.

가드레일이라는 임시방편

업계의 표준 대응은 AI 가드레일이었다. LLM의 입력과 출력을 걸러내고 감시하고 통제하는 보조 소프트웨어 층이다. 조직들이 생성 인공지능을 운용화하려는 경주 속에 가드레일 생태계는 빠르게 자라났다.

그러나 기존 솔루션 대다수를 들여다보면 공통점이 있다. 확률 분류기와 벡터 유사도 검색, 구문 스키마 검증에 기대고 있다. 이 도구들은 콘텐츠 조정에는 능하다. 욕설을 걸러내고 주제를 묶고 형식을 확인하는 일은 잘한다. 문제는 다른 곳이다. 미국 증권거래위원회(SEC), 금융산업규제당국(FINRA), 통화감독청(OCC)이 요구하는 복잡한 다변수 규제 제약을 강제하는 일에는 근본적으로 부적합하다. 거래 금액과 자본 한도와 마진 조건이 얹힌 명제는 분류의 대상이 아니라 증명의 대상이기 때문이다. 왜 부적합한지는 2장에서 가드레일의 세 계보를 놓고 뜯어본다.

입구에 수학을 세운다

논문이 내놓는 답은 패러다임의 전환이다. 확률을 더 정교하게 거르는 대신 형식 검증으로 건너가라는 것이다. 자연어로 쓰인 기관의 정책을 의존 타입 형식 언어 Lean 4로 번역해 컴파일해 둔다. 번역을 맡는 것은 Harmonic AI의 Aristotle 모델 같은 자동 형식화 도구로, 4장에서 자세히 본다. 에이전트가 제안하는 모든 행동은 수학적 추측으로 취급된다. Lean 4 커널이 그 추측이 사전에 컴파일된 규제 공리들을 만족함을 증명했을 때만 실행이 허가된다. 증명이 안 되면 행동은 정의상 일어나지 않는다. 반대도 성립한다. 증명된 행동은 막을 이유가 없다. 통과 기준이 ‘분류기가 안전하다고 본다’에서 ‘커널이 증명했다’로 바뀐다.

이 전환의 상대는 LLM이 아니다. LLM의 창의성은 그대로 쓴다. 바뀌는 것은 실행의 자리다. 확률이 문지기를 맡던 입구에 이진 판정의 수학을 세우는 것이다. 다음 장부터 이 전환이 왜 필요한지, 그리고 어떻게 작동하는지를 하나씩 연다.

DAY

02

가드레일의 세 계보



에이전트를 통제하려는 기술은 하루아침에 나온 것이 아니다. 세 번의 세대 교체를 거쳤고, 각 세대는 앞 세대의 한계에 대한 응답이었다. 결정론의 경직성은 확률을 낳았고, 확률의 흐릿함은 다시 증명을 불렀다. 계보를 아는 일은 단순한 역사 공부 아니다. 각 세대가 무엇을 해결했고 무엇을 남겼는지가 지금 무엇을 선택할지를 결정하기 때문이다. 이 장은 그 세 계보를 나란히 세워 놓고, 마지막 계보만이 금융 규제 요구와 문법이 같은 이유를 밝힌다.

| 옛것들의 미덕

1세대는 결정론적 자동화다. RPA 봇과 하드코딩된 워크플로우 엔진이 여기 속한다. 이들의 미덕은 정밀성이다. 정의해 둔 논리 경로를 수학적 정밀성으로 실행하고, 정의되지 않은 조건을 만나면 정지한다. 같은 입력은 같은 출력으로 이어지고, 결합이 생기면 재현되며, 재현되면 고친다. 감사와 디버깅의 관점에서 보면 꿈의 성질이다. 시스템이 무엇을 했는지 완벽하게 말해 준다. 규제 검사관 앞에서 이런 시스템은 떨어질 이유가 없다.

한계도 명확하다. 경직성이 비용이다. 정의 밖의 상황이 늘어날수록 경로를 새로 그려야 하고, 시스템은 변하는 시장을 따라가지 못한다. 자동화 범위를 넓힐 때마다 사람이 경로를 일일이 그려 주는 부담이 함께 자란다. 상품이 바뀌고 규정이 개정되고 시장이 닫히고 열릴 때마다 엔지니어가 불려 나온다. 유연성이 필요한 자리에 결정론은 무거워진다. 그 무게를 덜어 주려는 요구가 다음 세대를 낳았다.

| 확률의 필터

2세대는 그 경직성에 대한 응답으로 태어난 AI 가드레일이다. 조직들이 생성 인공지능을 운용화하는 경주와 함께 시장에는 다양한 가드레일 솔루션이 쏟아졌다. 생태계가 활발하다는 것 자체는 좋은 소식이다. 다만 활발함이 적합함을 보장하지는 않는다. LLM의 유연함을 통제망에도 접붙일 수 있다는 발상이다. 아이디어 자체는 자연스럽다. 언어를 이해하는 모델이라면 정책의 언어도 이해할 것이다. 그러나 비판적으로 들여다보면 주도권을 쥔 두 방법 모두 금융 규제의 요구와 어긋나 있다. 대표를 하나씩 본다.

NVIDIA NeMo Guardrails는 Colang이라는 전용 모델링 언어로 대화 흐름과 주제 경계를 정의하는 오픈소스 툴킷이다. 아키텍처는 본질적으로 확률적이다. 사용자나 에이전트가 프롬프트를 내놓으면 NeMo는 LLM을 불러 그 의도의 정규형을 만들고, 벡터 유사도 알고리즘으로 정규형을 미리 정의한 규칙과 매칭한다. 규칙을 코드로 못 박는 대신 의미의 근접함으로 묶는 방식이다. 여기에 구멍이 있다. 벡터 공간은 연속적이어서 적대적 섭동이 의미 거리를 쉽게 왜곡한다. 문장을 조금만 비틀어도 의미 공간에서의 위치가 옮겨 간다. 금지된 행동이 무해한 것으로 오분류될 수 있다. 평가는 LLM-as-a-judge 호출에 크게 의존하는데, 이것이 임계 경로에 수백 밀리초의 지연을 얹고 시스템 전체 처리량을 떨어뜨린다. 초당 수천 건이 오가는 실행 게이트에서 수백 밀리초는 선택이 아니라 배제다.

Guardrails AI는 방향이 다르다. RAIL 명세와 Pydantic 같은 파이썬 기반 라이브러리로 LLM 출력의 구문적 일관성을 강제한다. 매번 같은 판정을 내리고, 판정이 빠르다. 속도와 결정성은 확보한다. 그 장점은 진짜다. 문제는 그 장점이 지켜 주는 것이 형식뿐이라는 데 있다. 논리적으로 얕다. 논문의 표현을 빌리면 이렇다. Pydantic은 에이전트가 제안한 거래량이 정수임은 보장한다. 그 정수가 마진 사용 이력과 실시간 자본 제약과 결합했을 때 다층 규제 정책을 지키는지는 증명하지 못한다. 형식은 검사하지 내용의 논리는 검사하지 않는다. 규제 준수가 사는 곳은 정확히 결합의 자리인데도 그곳이 비어 있다. 이 장에서는 계보의 관점에서만 짚었다. 두 제품이 시장에서 실제로 어떤 자리에서 경쟁하는지, 형식 검증 진영이 어디에 서는지는 9장에서 상세 비교로 다시 만난다.

여기서 이 장의 핵심 논리가 나온다. 확률 필터가 99.9퍼센트의 위반을 막아도, 남은 수학적 가능성 때문에 '전속 통제' 기준을 충족하지 못한다. 회피 가능성이 수학적으로 존재하는 한, 기관은 절대 준수를 보장했다고 말할 수 없다. 보증서에 "99.9퍼센트"라는 단서를 붙일 수 없다는 뜻이다. SEC가 브로커딜러에게 요구하는 것은 확률적 안전망이 아니라 브로커 자신이 직접 그리고 독점적으로 쥔 통제다. 판정권을 확률적 모델에 넘겨 버린 순간 그 요건은 무너진다. 확률적으로 움직이는 LLM을 안전 필터로 배치하는 것 자체가 심각한 규제 취약점이라는 판단이 여기서 나온다.

I 증명의 시대

3세대는 이 기준을 받아들이고 출발한다. 형식 검증이다. 논문이 제안하는 Lean-Agent Protocol은 검증의 무게를 Lean 4 커널로 옮긴다. 벡터 매칭도 스키마 검사도 아닌 수학적 정리 증명으로 통과와 차단을 판정한다. 에이전트의 행동은 추측이고, 커널의 응답은 이진 판정이다. 증명됐으면 실행하고, 아니면 막는다. 모호한 중간값이 없다. 확률의 흐릿함 대신 증명의 흑백이 입구를 지킨다.

1세대가 잃었던 유연성은 어디로 갔을까. 사라진 것이 아니라 자리를 옮겼다. 경로를 그리는 사람의 손에서 정책을 증명으로 번역하는 모델의 능력으로 옮겨 갔다. 정책이 바뀌면 규칙 파일을 고치는 대신 새 정리를 컴파일한다. 결정론의 보장은 그대로 가져오면서 경직성의 비용만 내려놓는 설계다. 어떻게 그게 가능한지는 4장과 5장에서 확인한다.

여기에 자연의 속도 겹친다. Lean 커널의 판정은 마이크로초 단위다. 깊은 논리와 짧은 자연을 동시에 갖춘 조합은 초당 수 천 건이 오가는 고빈도 규제 실행 환경에서 생존할 수 있는 유일한 그림이다. 논리가 깊어도 느리면 문지기 자리를 못 맡고, 빠르지만 알으면 규제의 요구를 못 채운다. 두 축을 동시에 만족하는 것이 3세대의 실질적 의미다.

세 계보를 나란히 놓으면 그림이 정리된다.

판정 기준	RPA·워크플로우 엔진	NeMo Guardrails	Guardrails AI	Lean-Agent Protocol
결정성	결정적	확률적	결정적(구문 한정)	결정적
논리 깊이	고정 경로, 얕음	의도 매칭, 얕음	구문 검증, 얕음	정리 증명, 깊음
판정 지연	짧음	수백 밀리초	짧음	마이크로초
규제 적합성	경직성으로 부적합	확률성으로 부적합	논리가 얇아 부적합	전속 통제 충족

계보를 관통하는 축은 하나다. 무엇이 판정자인가. 1세대의 판정자는 경로고, 2세대는 확률과 구문이고, 3세대는 증명이다. 금융 규제의 언어는 세 번째와 문법이 같다. 규정은 조건들의 논리식이고, 준수는 그 식의 증명이기 때문이다. 경직된 경로는 시장의 변화를 못 따라가고, 확률의 필터는 규제의 보장을 못 채운다. 증명만이 유연성과 보장을 동시에 품는 자리에 있다. 물론 증명 기반 가드레일이 실제로 빠르게 돌아갈 수 있는지, 그리고 자연어 정책을 정리로 옮기는 일이 현실적인지는 별개의 질문이다. 그 물음에 답하는 것이 이어지는 장들의 몫이다. 다음 장에서 이 판정자가 어떤 도구 위에 서는지 들어간다.

DAY

03

Lean 4의 문법



이 장은 원문이 몇 문단으로 압축해 넘긴 형식 검증의 배경을 한국 독자용 강의로 넓혀 쓴 것이다. 개념의 골격과 사실 — Lean 4의 성질, 커널의 역할, Lean4Lean과 AWS의 사례 — 은 원문과 그 인용 문헌에서 가져왔고, 비유와 설명의 순서는 필자의 각색이다.

수학으로 짓는 울타리

형식 검증이 무엇인지부터 정하자. 테스트는 샘플을 돌려 본다. 형식 검증은 속성이 모든 가능한 상태 공간에서 성립함을 기계적으로 보장한다. “이 입력에서 괜찮았다”가 아니라 “어떤 입력에서도 괜찮다”를 증명한다. 두 접근의 격차는 입력의 개수가 늘어날수록 벌어진다. 테스트는 아무리 많이 돌려도 상태 공간의 일부만 방문한다. 증명은 상태 공간 전체를 한 번에 다룬다.

이 방법론은 안전이 생명인 하드웨어와 항공우주 공학에서 다듬어졌다. 비행기가 하늘 위에서 코드를 패치할 수 없기 때문이다. 의료 기기와 원자로 제어기도 같은 길을 걸었다. 결함을 실행 후에 고칠 수 없는 시스템은 실행 전에 수학으로 결함을 걷어 내는 수밖에 없다. 금융 실행 게이트가 이 목록에 합류하는 이유도 같다. 체결된 주문은 하늘 위의 비행기처럼 취소할 수 없다.

그 도구 중 하나가 Lean 4다. Lean 4는 의존 타입(dependently typed) 형식 언어다. 타입이 값에 의존한다는 뜻이다. 일반 프로그래밍 언어의 타입이 “정수다”라고 말한다면, 의존 타입 언어는 “일일 자본 한도보다 작은 거래액이다”처럼 명제를 타입 자리에 놓는다. 명제가 타입이면 증명은 그 타입의 값이 된다. 프로그램을 짜는 일과 정리를 증명하는 일이 같은 행위로 겹친다. 컴파일러가 타입을 확인한다는 것은 곧 증명을 확인한다는 것이다.

Lean 4의 이면에는 근거 이론인 귀납적 구성 계산법(Calculus of Inductive Constructions, CIC) 규칙 체계가 깔려 있다. 이 규칙 체계가 무엇을 만들 수 있고 무엇을 증명할 수 있는지를 정의한다. 그 위에는 Mathlib라는 종합 수학 라이브러리가 쌓여 있다. 대학 수학 교과서의 내용이 기계 검증된 형태로 축적된 저장소다. 수학자들이 실제 연구에 쓰는 도구라는 점이 중요하다. 장난감 검증기가 아니라 최전선의 도구다.

커널이라는 심판

Lean 생태계의 신뢰가 서는 지점은 구조의 분리다. 증명을 만드는 일과 증명을 검사하는 일이 갈라져 있다. 사람이든 인공지능이든 증명을 생성하는 과정은 느리고 시행착오가 많다. 시도하고, 틀리고, 고치는 일의 연속이다. 그러나 완성된 증명을 검사하는 일은 빠르고 기계적이다. 검사기 자체도 작다. Lean의 신뢰 커널은 고성능 C++로 쓴 최소한의 원시 연산에만 의존한다. 신뢰해야 할 코드의 양, 다시 말해 신뢰 기반을 극한까지 좁히는 설계다. 검사관이 작을수록 검사관 자체가 틀릴 확률이 줄어든다. 신뢰를 넓게 뿌리는 대신 한 점에 모으는 전략이다.

이 분리가 왜 실무적으로 중요한지는 5장에서 실측 속도로 확인한다. 정책을 형식화하고 증명을 생성하는 무거운 일은 비동기로 미리 해 두면 되고, 런타임에는 가벼운 검사만 돌리면 되기 때문이다. 생성은 느려도 되지만 검사는 매 행동마다 돌아간다. 무거운 일과 가벼운 일의 시간대를 갈라 놓는 것, 그것이 형식 검증을 실시간 시스템에 들여보내는 열쇠다.

검증자를 검증하는 질문도 성립한다. Lean4Lean(arXiv 2403.14064)은 Lean 타입검사기 자체를 Lean으로 구현하고 그 정확성을 기계적으로 검증한 연구다. 심판이 공정한지를 다른 심판에게 심사받는 셈이다. 커널을 맹신하지 않고 커널의 정확성 정리까지 증명하려는 시도다. 검증의 사슬이 바닥까지 이어지는지를 묻는 형식 방법론 특유의 성실함이 담긴 사례다.

이 접근이 이미 상용 규모를 통과했다는 증거도 있다. 아마존웹서비스(AWS)는 오픈소스 인가 정책 언어 Cedar의 핵심 구성요소 — 평가기, 권한부여기, 검증기 — 를 Lean 4로 형식 모델링하고 검증한다. AWS는 이 개발 방식을 검증 주도 개발(verification-guided development)이라 부른다. 코드를 짜고 테스트를 나중에 붙이는 대신, 증명할 정리를 먼저 세우고 구현이 그 정리를 만족하도록 개발을 이끈다. 예컨대 forbid 정책이 permit 정책보다 평가 순서와 관계없이 항상 우선한다는 정리를 증명해 두면, 방대한 생산 인가 판정이 그 증명 위에서 돌아가도 접근 제어가 규칙을 어기지 않음을 보장받는다. 클라우드 최대 규모의 인프라가 형식 검증의 무게를 견디고 있다는 실증이다. 9장에서 이 사례를 기업 도입의 선례로 다시 연다.

I 증명의 구멍, sorry

형식화는 처음부터 완벽하지 않다. 복잡한 금융 정책을 옮기는 초안은 논리적 단계 일부를 증명하지 못한 채 남길 수 있다. Lean은 그 구멍을 `sorry`라는 전술로 표기한다. `sorry`는 “여기가 아직 증명되지 않은 에피스테믹 갭이다”라는 선언이다. 구멍을 숨기지 않고 이름표를 붙여 둔다. 컴파일은 통과하되 증명은 불완전하다는 상태가 명시적으로 남는다. 거짓으로 완성된 것처럼 보이는 것보다 훨씬 정직하고, 훨씬 다루기 쉽다.

이 표기가 프로토콜에서 중요한 이유는 그것이 채워지기 때문이다. 원문이 다루는 Aristotle 모델은 하나의 정리 안에 남은 여러 개의 `sorry`를 스스로 추론해 메울 수 있다. 고수준 규제 지시와 기계 실행에 필요한 세밀한 논리 게이트 사이의 간격을 자동으로 잇는다. 구멍을 찾는 일은 정책 번역의 몫이고, 구멍을 메우는 일은 증명 탐색의 몫이다. 그 과정은 4장에서 천천히 본다.

한도 초과 주문을 막는 정리가 어떤 모양인지 의사 코드로 맛보자. 실제 문법과 다를 수 있다.

```
-- 의사 코드: 개념 소개용이며 실제 Lean 4 문법과 다를 수 있다
theorem order_within_capital
  (order : TradeOrder) (env : MarketState)
  (h_limit : order.value ≤ env.daily_capital_limit) :
  order.execution_allowed = true
```

거래 주문 `order`가 시장 상태 `env`의 일일 자본 한도 안에 있으면 실행이 허가된다는 명제다. 정확한 문법을 지금 알 필요는 없다. 기억할 것은 하나다. 규정이 주석이나 코드가 아니라 정리로 존재하기 시작했다는 것. 정책이 이런 정리로 컴파일돼 정책 환경에 쌓이면, 에이전트의 행동은 매번 이 정리의 자리에 대입되는 추측이 된다. 커널이 그 추측을 받아 들이느냐 마느냐가 곧 허가냐 차단이냐다.

DAY

04

Aristotle과 자동 형식화



“일일 가용 자본의 10퍼센트를 넘는 거래는 하지 말라.” 규정 준수 담당자가 이 한 줄을 내려 놓으면, 기계는 그것을 무엇으로 바꿔야 할까. 확률적 에이전트가 읽는 조언으로는 부족하다. 실행 직전에 예와 아니오를 가르는 기계 판정이 필요하고, 그러려면 자연어를 수학의 문법으로 옮기는 번역가가 있어야 한다. 이 장의 주인공이 바로 그 번역가다. 실리콘밸리 스타트업 Harmonic AI가 만든 Aristotle은 신경망의 언어 능력과 기호 논리의 엄밀함을 한 몸에 붙인 모델로, 논문의 Lean-Agent Protocol은 이 모델 위에 세워진다.

신경과 기호의 결합

Aristotle을 한마디로 말하면 신경-기호(neural-symbolic) 모델이다. 우리에게 익숙한 대형 언어 모델은 다음 토큰을 하나씩 예측하며 문장을 이어 간다. 유창하지만, 문장이 참인지는 아무도 보증하지 않는다. Aristotle은 구조적으로 다른 길을 택했다. 자동 정리 증명과 ‘형식 검증된, 환각 없는 추론’을 목표로 설계되었고, 모델이 내놓는 추론 단계는 Lean 4라는 형식 언어로 검사를 통과해야 완성된다.

그 실력을 보여 준 것이 국제 수학 올림피아드다. 2025년 대회에서 Aristotle은 금메달에 해당하는 성적을 냈다. 여섯 문제 가운데 다섯 문제를 사람의 손을 거치지 않고 풀었고, 그 해답이 Lean 4 기계 검증을 통과했다는 점이 다르다. 시험 점수가 아니라 ‘증명이 컴파일된다’는 사실이 성적표였다.

내부는 세 개의 부품으로 이루어져 있다. 첫째는 연속적인 Lean 증명 탐색 시스템이다. 증명의 공간을 체계적으로 헤집으며 완성된 증명을 찾는다. 둘째는 비형식 추론 엔진이다. 사람이 하듯 자연어로 중간 보조정리를 만들고, 그것을 형식 코드로 옮긴다. 큰 정리를 한 번에 증명하지 않고 잘게 쪼개는 전략이다. 셋째는 기하 전용 솔버다. 도형 문제는 일반 추론과 결이 달라 별도 엔진이 맡는다.

훈련 방식도 특이하다. Aristotle은 강화 학습 루프 안에서 Lean 4 컴파일러를 보상 신호로 쓴다. 보상을 사람 피드백이 아니라 컴파일러가 준다. 컴파일러는 실망하지 않고, 아부하지 않고, 규칙(CIC, 귀납적 구성 계산법)에 맞지 않으면 무조건 거절한다. 오류 없는 고속 채점관이 있는 한, 모델은 자기 추론을 계속 수학의 잣대에 대조할 수밖에 없다.

자연어를 수학으로 옮기는 일

Lean-Agent Protocol이 Aristotle을 고용한 결정적 이유는 자동 형식화(auto-formalization) 능력 때문이다. 자동 형식화는 비정형적인 자연어 개념을 엄밀한 형식 코드로 번역하는 작업이다(형식화 연구 자체는 arXiv 2406.01940 따위의 선행 연구에서 활발히 진행되어 왔다). 번역이 이루어지지 않으면 “자본의 10퍼센트”라는 말은 컴퓨터가 집행할 수 없는 문장일 뿐이다.

Aristotle은 이 분야에서 이미 실적을 쌓았다. Lean의 종합 수학 라이브러리인 Mathlib에 새로운 기여를 했고, 오랫동안 풀리지 않던 에르되시 문제 1026과 728의 증명을 형식화했으며, 확립된 교과서에서 미세한 논리 오류를 찾아 냈다. 교과서가 틀렸고 모델이 맞았다는 말이 아니라, 형식화를 시도하자 교과서 서술의 논리적 빈틈이 드러났다는 뜻이다. 자연어는 모호함을 숨기는 데 능하고, 형식화는 그 모호함을 적발하는 데 능하다.

증명의 구멍, sorry를 채우는 기계

그렇다면 번역은 언제나 완벽할까. 아니다. 복잡한 금융 정책을 Lean으로 옮기면 초안에는 증명되지 않은 구멍이 남는다. Lean은 이 구멍을 sorry라는 전술로 표시한다. “여기는 아직 증명 못 했다”는 표식이며, 3장에서 본 에피스테믹 갭의 공식적 자리다.

주목할 것은 Aristotle이 이 구멍을 스스로 메운다는 사실이다. 하나의 정리 안에 남은 여러 개의 sorry를 자율적으로 추론해 채워 넣는다. 이것이 왜 금융에 중요한가. 고위 규제 지시는 사람의 언어로 되어 있고, 기계가 실행할 논리 게이트는 원자적으로 잘게 나뉘어 있기 때문이다. “자본 한도를 넘지 말라”는 지시와 $\text{Trade_Value} \leq \text{Capital_Limit}$ 라는 게이트 사이에는 여러 단계의 논리적 사다리가 놓여 있다. Aristotle이 각 단계의 증명을 매워 주지 않으면 정책은 실행 계층에 닿지 못한다.

스스로 고치는 번역, 자기수리 사이클

번역의 충실도는 언제나 묻는 질문이다. 번역기가 실수하면 규정의 본뜻이 코드에서 빠지는 것이 아닌가. 논문이 주는 답은 냉정하다. 완벽한 첫 생성을 기대하지 말라. 대신 Lean 환경의 결정론적 구속력이 자기수리(self-repair)를 강제한다고.

실제 사례가 소개된다. Aristotle의 비형식 추론 엔진이 복잡한 정리를 형식화하다가, '엄격하게 감소하는 수열'과 '약하게 감소하는 수열', 그리고 '어느 시점에만 감소하는 수열'이라는 서로 다른 정의를 한데 섞은 문장을 만들어 냈다. 사람의 눈에는 그럴듯한 서술이다. 일반적인 생성 파이프라인이라면 이 오류는 그대로 통과해 제품 속으로 들어간다. 아무도 문장의 참여를 검사하지 않기 때문이다.

Aristotle의 아키텍처에서는 흐름이 다르다. 비형식 증명이 만들어지면 곧바로 Lean 4 커널에 형식화물을 제출한다. 커널은 결정적으로 거절한다. 엄격 감소와 약한 감소를 섞은 코드는 어중간한 수열을 반례로 내밀며 컴파일 오류를 뱉는다. 오류 메시지는 구체적이라, 어떤 명제가 어떤 반례에 무너지는지까지 짚어 준다. Aristotle은 이 오류 상태를 흡수해 논리의 빈틈을 고치고, 컴파일이 성공할 때까지 수리를 반복한다.

이 사이클의 산물은 '그럴듯한 답'이 아니라 '형식화된 문제 명제 기준으로 정확히 증명된 답'이다. 최종 출력이 참임이 기계 검증으로 닫히므로, LLM이 그럴듯한 준수 서술을 지어내는 병 — 논문은 이를 환각 컴플라이언스(hallucinated compliance)라고 부른다 — 는 구조적으로 중화된다. 규정을 지킨 것처럼 보이는 문장이 아니라, 규정을 지켰음을 증명할 수 있는 코드만 다음 단계로 넘어온다.

여기서 얻는 교훈은 의외로 개발자에게 익숙한 것이다. 우리는 이미 타입 오류를 고치며 코드를 다듬는다. 컴파일러의 거절은 실패가 아니라 정보다. Aristotle은 이 일상을 자연어 정책 번역에 그대로 이식한 것이다. 다음 장에서는 이렇게 번역된 정책이 실행 시점에 어떤 문지기를 통과하는지, 그 전체 순환 구조를 따라 가 본다.

DAY

05

결정론적 승인 루프



4장까지 준비물을 갖쳤다. Lean 4라는 문법(3장)과, 자연어 정책을 그 문법으로 옮기는 번역가 Aristotle(4장). 이 장에서는 준비물을 조립해 실행 시스템을 완성한다. 논문이 제시하는 것은 상태 기계로 돌아가는 요청-응답 루프다. 거래 에이전트가 아무리 확률적으로 생각하더라도, 그 출력은 API 실행 직전의 결정론적 병목을 강제로 통과한다. 통과 조건은 하나다. 커널이 증명하면 간다, 못 하면 멈춘다.

루프는 시간의 결이 다른 두 국면으로 나뉜다. 정책을 미리 만들어 두는 비동기 국면과, 요청이 들어온 순간 판정하는 동기 국면이다.

정책을 미리 짓는 시간 — 비동기 구성

첫 국면은 서두르지 않는다. 규정 준수 담당자가 기관의 거래 한도와 리스크 파라미터를 일상어로 쓴다. “일일 가용 자본의 10퍼센트를 초과하는 거래를 집행하지 말라” 같은 문장이다. 특별한 표기법이 필요 없다. 정책의 저자는 여전히 사람이다.

다음으로 Aristotle이 이 문서를 처리한다. 비형식 추론 엔진이 문장을 해석하고, 증명 탐색 시스템이 그 제약을 Lean 4 코드로 자동 형식화한다. 생성된 코드는 Lean 커널이 컴파일하고, 논리적 모순이 있으면 4장의 자기수리 사이클이 반복해서 고친다. 컴파일이 성공하면 산출물 — 공리와 정리, 정의 — 는 정책 환경(Policy Environment)이라 불리는 불변 저장소에 쌓인다. 불변이라는 말이 무게를 갖는다. 한 번 검증된 정책은 몰래 고쳐질 수 없고, 바꾸려면 처음의 번역-컴파일-검증 고리를 다시 통과해야 한다.

이 국면이 비동기인 까닭은 간단하다. 번역과 증명 생성은 무겁고 느릴 수 있는 작업인데, 시장이 열려 있는 동안 하려면 늦는다. 그래서 미리, 거래가 없는 시간에 해 둔다.

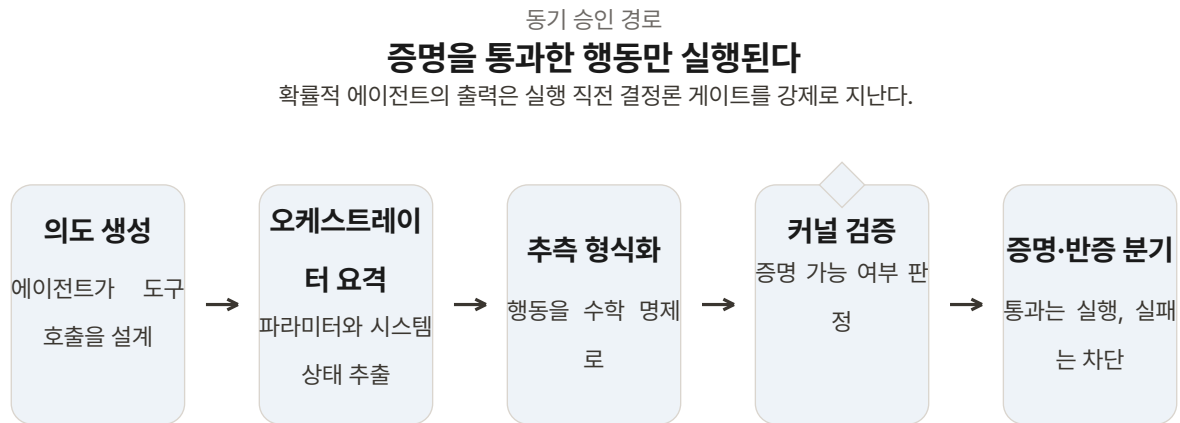
요청을 걸러내는 시간 — 동기 승인 5단계

두 번째 국면은 실시간이다. 포트폴리오 리밸런싱 에이전트가 실시간 시세를 읽고 실행 의도를 만든다. 예를 들어 `execute_trade(symbol="AAPL", volume=50000, type="market")` 같은 API 호출이다. 이 순간부터 다섯 단계가 순서대로 진행된다.

첫 단계는 의도 생성이다. 에이전트가 특정 도구 호출을 설계한다. 여기까지는 에이전트의 자유다. 둘째 단계에서 오케스트레이터가 이 호출을 실행 환경에 닿기 전에 가로챈다. 요청을 토큰으로 분해해 파라미터와 현재 시스템 상태 변수 — 가용 자본, 최근 변동성 지수 — 를 추출한다. 셋째 단계에서 오케스트레이터는 정책 환경에 저장된 Lean 정리들에 이 값들을 대입해, 에이전트의 행동 하나를 하나의 수학적 추측(conjecture)으로 만든다. “이 거래는 모든 정책 정리를 만족하는가”라는 명제다. 넷째 단계에서 추측은 Lean 4 타입검사기에 제출되고, 커널이 증명을 시도한다.

다섯째 단계는 판정의 이행이다. 여기서 루프는 이진값으로 닫힌다. 상태 A — 증명 성공이면 커널은 참(True)을 반환하고, 게이트웨이가 열려 원래 API 호출이 실행 계층으로 흘러간다. 상태 B — 증명 실패면 거짓(False)이 반환된다. 거래 볼륨이 자본 정리를 초과했다는 식의 반증이고, 행동은 결정적으로 차단된다. 이때 시스템은 감사 로그용으로 형식 오류 추적을 남긴다. 왜 막혔는지가 수학의 언어로 기록되는 것이다.

다섯 단계에서 에이전트의 몫과 시스템의 몫이 정확히 같린다. 에이전트는 첫 단계에서 생각할 자유를 갖는다. 의도를 만드는 일에는 확률이 개입해도 좋다. 그러나 둘째 단계 이후는 전부 오케스트레이터와 커널의 영토다. 생각하는 주체와 실행하는 주체를 이렇게 분리해 놓으면, 모델이 아무리 이상한 제안을 하더라도 그것이 세상에 닿는 길은 하나뿐임이 보장된다. 확률 필터가 ‘위험해 보이는 입력을 골라 막는’ 방식과 결정적으로 다른 지점이다. 필터는 막히지 않고 통과한 것을 다시 검토하지 않지만, 이 루프는 통과 자체가 증명의 결과물이다. 다만 통과 뒤에 어떤 재심사도 필요 없다고까지 읽는 것은 필자의 확장 해석이다. 원문의 1단계 로드맵은 그림자 모드에서 인간 컴플라이언스 리뷰와의 대조를 명시적으로 유지한다(14장).



편집 요약: 본문의 동기 5단계 승인 흐름을 재배열한 도식이다.

I 느리다는 편견 — 자연의 논증

이쯤에서 누구나 묻는다. 실행 경로마다 정리 증명을 돌리면 느리지 않은가. 형식 검증은 계산 비용이 크다는 것이 오랜 통념이고, 고빈도 거래 같은 밀리초의 세계에는 쓸 수 없다는 비판이 그 근거였다.

논문의 답은 증명 생성과 증명 검사의 분리에 있다. 무거운 일 — 증명을 만들고 정책을 형식화하는 일 — 은 전부 비동기 구성 국면에서 끝난다. 동기 실행 시점에 커널이 하는 일은 사전 컴파일된 바이너리를 대상으로 한 타입검사와 증명 검증뿐이다. 검사만큼은 Lean의 신뢰 커널이 최소한의 고성능 C++ 프리미티브로 구현되어 극도로 최적화되어 있다.

실측 근거도 있다. 아마존웹서비스(AWS)는 접근 제어 정책 언어 Cedar를 Lean 4로 형식 검증하는데, 차등 테스트 환경에서 형식화된 접근 제어 입력 하나를 Lean 모델로 평가하는 데 평균 5마이크로초가 걸렸다. 같은 입력을 고도로 최적화된 Rust 구현으로 평가한 평균은 7마이크로초였다. 즉 형식 모델의 평가가 손으로 다듬은 시스템 코드보다 빨랐다.

여기에 최근의 도구 진보가 더해진다. Lean 4 생태계에 도입된 자동화 전술 grind는 SMT(모듈로 이론 만족성) 풀이 능력을 증명기 안으로 끌어온다. 그래서 오케스트레이터가 `Trade_Value ≤ Capital_Limit` 같은 단순 산술 제약을 제출하면 커널은 서브밀리초에 답한다. 결론은 명쾌하다. 형식 검증은 실행 임계 경로의 병목이 아니다. 느린 학문이 아니라, 무거운 부분을 미리 하고 가벼운 부분만 현장에서 하는 공학이다.

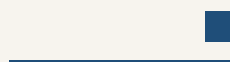
측정 단위를 체감해 보자. 마이크로초는 밀리초의 천분의 일이다. 사람의 눈이 자연이라고 느끼는 문턱이 대략 백 밀리초인데, 커널의 판정은 그 문턱에서 이만 배즈 아래에서 끝난다. 수백 밀리초를 엮는 LLM-as-a-judge 방식과 비교하면 차이는 자릿수 단위다. 도입부에서 확률 필터의 속도를 의심했던 독자라면, 오히려 판사 역할을 하는 언어 모델 쪽이 훨씬 무겁다는 사실을 새겨 둘 만하다.

정리해 보자. 비동기 국면이 정책을 수학으로 굳히고, 동기 국면이 매 행동을 그 수학에 대어 본다. 확률적 사고와 결정론적 실행이 만나는 접점은 오직 하나, 커널의 판정뿐이다. 다음 장부터는 이 구조가 실제 규제 문장들 — 시장 접근 규칙, 모형 리스크 관리, 설명 의무 — 과 어떻게 맞물리는지 확인한다.

DAY

06

규제 매핑 — 한국 금융으로



두 사건이 만든 규칙

1987년 10월, 시장은 하루 만에 무너졌다. 프로그램 매매가 하락을 증폭시켰고, 규제 당국은 알고리즘이 시장에 접속하는 방식 자체를 감독의 대상으로 삼기 시작했다. 25년 뒤인 2012년에는 Knight Capital이 무너졌다. 배포 실수로 살아남은 오래된 코드를 재활용한 매매 알고리즘이 45분 동안 4억 4천만 달러를 잃었고, 회사는 존속 위기에 몰렸다. 확인되지 않은 알고리즘이 시장에 직접 접속해 있었다는 사실이 문제의 핵심이었다.

원문은 이 두 사건을 거쳐 미국 증권거래위원회(SEC)의 규칙 15c3-5, 이른바 시장 접속 규칙(Market Access Rule)이 도입됐다고 서술한다. 연대는 짚고 넘어가자. 규칙의 채택은 2010년이고 나이트 캐피탈 사고는 그 2년 뒤에 터졌다. 사건이 규칙을 낳지는 않았지만, 규칙이 겨눴던 표적 — 시장에 직접 닿는 알고리즘 주문의 사전 통제 — 은 두 사건이 보여 준 재앙의 형태와 정확히 맞닿아 있다. 규칙의 뼈대는 이렇다. 브로커딜러가 고객이나 자사 알고리즘에 시장 접속 수단을 줄 때, 재무적·규제적 리스크를 관리하도록 합리적으로 설계된(reasonably designed) 위험 관리 통제와 감독 절차를 수립하고 문서화하며 유지할 것을 요구한다. 요건은 구체적이다. 사전에 정해 둔 신용·자본 한도를 넘는 주문은 들어오기 전에 차단한다. 가격이나 수량 파라미터를 벗어난 요청은 거절해 오주문을 방지한다. 그리고 여기에 결정적인 문장이 하나 붙는다. 이 통제들은 브로커딜러의 '직접적이고 전속적인 통제' 아래 놓여 있어야 한다.

'전속 통제'라는 벽

여기서 기존 AI 가드레일의 한계가 정확히 드러난다. 확률 분류기를 얹은 필터가 비준수 주문의 99.9퍼센트를 막았다고 하자. 남은 0.1퍼센트는 통계적으로는 작아 보이지만, 수학적으로는 여전히 가능한 사건이다. 탈출 가능성이 존재하는 순간 그 통제는 '전속'이 아니다. LLM은 본질적으로 비결정론적이다. 연속적인 벡터 표현 위에서 확률적 샘플링으로 출력을 고르기 때문에, 같은 입력이 다른 행동을 낳을 수 있다. 그러나 규제가 요구하는 것은 높은 확률의 차단이 아니라 원천 봉쇄다. 규제 준수 언어가 '가능성의 크기'가 아니라 '가능성의 부재'라는 점이 바로 그 벽이다. 확률적 이차 필터를 안전 장치로 세우는 것은 이 기준 앞에서 심각한 규제 취약점이 된다.

증명으로 쓰는 한도

Lean-Agent Protocol의 응답은 자본 한도와 가격 파라미터를 불변의 Lean 4 공리로 형식화하는 것이다. 공리가 컴파일 되면 정책 환경에 저장되고, 에이전트의 실행 계층은 수학 논리에 묶인다. 검증 커널은 의도를 해석하지 않고 확률을 추정하지도 않는다. 이진 진리만 계산한다. 에이전트가 크기 파라미터가 한도를 넘는 주문을 제안하면 증명은 컴파일에서 실패하고, 그 주문은 게이트웨이 수준에서 결정적으로 거절된다. 이른바 '단단한' 가드레일이 이것이다. 그리고 규제 당국에 눈에 보이는 약속이 하나 생긴다. 오류 주문은 시장에 물리적으로 도달할 수 없다는 사실을 수학적으로 증명해 보일 수 있다. 에이전트 자동화가 시장 접속 규칙의 가장 엄격한 해석과 나란히 놓이는 순간이다.

모델 리스크 관리의 재시행

시장 접속 다음의 과제는 조직 차원의 모델 관리다. 통화감독청(OCC)의 고시 2011-12(SR 11-7)는 모델 리스크 관리(MRM) 프레임워크의 기초가 됐다. 모델의 개발, 구현, 지속 검증, 거버넌스를 촘촘히 묶는 체계다. 그런데 전통적 MRM은 모델이 결정론적 개체라는 가정 위에서 있었다. 입력을 해석할 수 있고 출력이 예측 가능하다는 가정이다. 생성형 AI와 자율 에이전트는 이 가정을 부순다. 높은 불투명성, 확률적 추론, 정적 검증 주기를 우회하는 자율 의사결정 루프가 조직 안으로 들어온다.

금융산업규제기구(FINRA)의 규칙 3110은 회원사가 증권 법규 준수를 위해 관련자와 시스템의 활동을 감독하는 체계를 세우고 유지할 것을 요구한다. 규정시정서 24-09와 2026년 연례 감독 보고서에서 FINRA는 자사 규칙이 기술 중립임을 분명히 했다. 생성형 AI 도구를 도입하면 그 도구도 인간 커뮤니케이션과 전통적 의사결정 시스템에 적용되는 것과 같은 엄격함으로 감독해야 한다는 뜻이다. 도구가 새로워도 감독의 기준은 낮아지지 않는다.

생각과 행동의 분리

원문이 이 장들에서 뽑아 낸 원리는 단순하고 강력하다. 확률적 '생각'과 결정론적 '행동'의 분리다. 에이전트의 LLM이 시장을 읽고 전략을 짜는 것은 생각의 영역에 둔다. 그 생각이 실행 API로 넘어가는 순간만 Lean 커널이라는 문지기가 검문한다. 어떤 에이전트 과제든 증명을 통과하지 못하면 실행되지 않는다. 감독관 앞에서 기관이 보여 줄 수 있는 그림은 이렇다. 바닥의 LLM이 어떤 확률적 출력을 내놓더라도, 감독 체계 전체는 인간의 통제 아래 100퍼센트 예측 가능하게 동작한다. LLM의 불투명성과 FINRA·OCC가 요구하는 투명성 사이에 프로토콜이 구조적 다리를 놓는 셈이다. 혁신과 거버넌스가 서로를 희생하지 않는 구조가 나온다.

한국 금융 감독에 비추어 — 필자의 각색

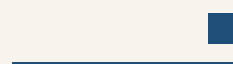
지금부터는 필자의 각색이다. 미국의 규칙 번호를 그대로 옮길 수는 없지만, 위의 논리는 한국 금융 감독의 지형에도 그대로 접힌다. 한국의 전자금융감독규정과 정보시스템 감독 체계는 오래전부터 사고를 막는 통제와 내부통제 구조를 요구해 왔다. 이용자별 이용 한도 설정, 이상 거래 탐지와 차단, 시스템 변경 시의 사전 검증과 사후 점검 같은 요구는, 제도의 방향으로만 놓고 보면 15c3-5가 묻는 것과 같은 질문이다. 누가 시장과 시스템에 접속하는지, 그 접속이 사고로 이어지지 않도록 누가 어디서 막는지. 한국판 시장접속 통제의 뼈대는 이미 서 있는 셈이다.

한국 감독 당국도 AI에 대한 시선을 빠르게 좁히고 있다. 금융 AI 챗봇의 고객 응대 책임 소재, 로보어드바이저의 알고리즘 투자 권유에 대한 검증과 설명 의무를 둘러싼 논의가 깊어지는 중이다. 이 흐름에 원문의 원리를 대입하면 다음과 같이 정리된다. AI가 내리는 권유와 실행은 사람이 세운 한도 안에서만 나가야 하고, 그 한도는 확률적 추정이 아니라 검증 가능한 규칙으로 고정돼야 하며, 차단이나 권유의 사유는 감독자와 고객이 읽을 수 있는 형태로 남아야 한다. 조문 번호를 인용할 수준의 확정 근거는 아직 없지만, 제도가 걷는 방향 자체는 분명하다. 통제의 주체는 도구가 아니라 기관이어야 하고, 그 통제는 검증 가능한 형태로 문서화돼야 한다. 에이전트가 스스로 주문을 넣고 계약을 체결하는 시대에 이 원칙은 오히려 더 무거워진다. 그리고 이 장이 남긴 질문 하나가 다음 장의 출발점이 된다. 그 검증 가능한 차단을 정작 당사자에게 설명해야 한다면, 무엇을 보여 줄 것인가.

DAY

07

설명할 권리와 역방향 형식화



법이 요구하는 설명

알고리즘이 사람에게 불리한 결정을 내렸다면, 그 결정에는 이유가 따라야 한다. 미국의 공평신용기회법(ECOA)과 공정신용보고법(FCRA) 아래에서 채권자는 불리 조치(adverse action)를 취할 때 구체적이고 정확하며 이해하기 쉬운 사유를 소비자에게 제공할 법적 의무가 있다. 소비자금융보호국(CFPB)은 복잡한 블랙박스 알고리즘이나 AI 모델을 쓴다고 해서 이 투명성 요구가 면제되지 않는다고 명시적으로 밝혔다. 유럽은 더 나아간다. GDPR과 새로 제정된 EU AI Act는 자동화 의사결정의 대상이 된 개인에게 그 판단 논리에 대한 의미 있는 설명을 요구할 권리를 준다. 이른바 '설명할 권리(Right to Explanation)'다. 기술의 정교함이 법의 등급을 올려 주지는 않는다. 금융 서비스에서 AI를 배치할 때 가장 복잡한 규제 과제 하나가 바로 여기서 나온다.

이 의무가 무거운 이유는 독자가 다양하기 때문이다. 대출 거절을 통보받는 소비자, 모델의 판정을 검증하는 감사인, 감독 보고서를 읽는 규제 당국은 같은 결정을 서로 다른 언어 수준에서 요구한다. 설명 의무는 이 세 독자를 모두 만족시킬 것을 요구한다. 어느 한쪽에 맞춘 통지는 다른 쪽에서 결함이 된다. 소비자에게는 너무 어렵고, 감사인에게는 너무 빈약한 설명은 두 번 실패한 설명이다.

컴파일러 오류는 통지문이 아니다

Lean-Agent Protocol은 차단 자체는 완벽하게 해낸다. 문제는 차단 다음이다. 증명이 실패하면 시스템이 손에 쥐는 것은 고도로 기술적인 의존 타입 수학의 오류 트레이스, 다시 말해 컴파일러 오류다. 감사 로그에는 최고의 기록이지만, 대출을 거절당한 소비자나 비기술 감사인에게 형식 논리 컴파일러 오류를 그대로 보여 주는 것은 법적으로 불충분하다. 쉬운 언어로 공개하라는 요구를 위반한다.

여기서 간과하기 쉬운 것은 이 간극의 성격이다. 컴파일러 오류를 사람이 읽기 좋게 다듬는 일을 미용 문제로 취급하면 법을 벗어난다. 불리 조치 통지는 소비자가 다음 행동을 결정할 수 있을 정도의 구체성을 요구한다. 사유가 모호하거나 기술적 은어에 묻히면 통지한 것이 아니라 통지를 회피한 것이 된다. 감사인의 입장도 비슷하다. 기관의 통제가 실제로 작동했음을 조직 바깥에서 입증하려면, 수학의 언어와 검증 가능한 관계를 유지하는 번역이 필요하다. 차단의 완성도와 통지의 품질이 별개의 과제라는 지점에서, 원문은 해법을 꺼낸다. 역방향 자동 형식화(reverse auto-formalization), 줄여서 역번역(back-translation)이다. 경직된 수학적 실패 상태를 규정 준수형 자연어 불리조치 통지로 되돌려 번역하는 작업이다.

실패의 좌표를 찍는 기술

거꾸로 번역하려면 먼저 어디서 무엇이 실패했는지 정확히 알아야 한다. 이 능력의 근거가 Herald 데이터셋(arXiv 2410.10878)과 NL2Lean 강화학습 프레임워크다. Herald는 정적 분석 도구로 Lean 4 환경에서 구조화된 메타데이터 — 정리 선언, 의존 관계, 정확한 증명 상태 — 를 추출할 수 있음을 보여 준다. 게이트웨이에서 증명이 실패하면 시스템은 거절을 유발한 정확한 코드 줄과 특정 논리 전술을 격리한다. 예컨대 에이전트가 제안한 행동이 `Debt_to_Income < 0.43` 공리를 통과하지 못했다는 식으로 좌표를 찍는다. 이어서 NL2Lean(EMNLP 2025)이 검증한 접근을 붙인다. 이렇게 격리된 증명 상태를 흡수하도록 파인튜닝한 전용 번역 모델이, 검색 증강 생성(RAG) 파이프라인을 이용해 실패의 정확한 단계별 비형식 설명을 만들어 낸다.

환각이 아닌 직역

이 설계의 요점은 설명조차 결정론의 사슬 위에 묶여 있다는 것이다. 소비자와 감사인에게 제공되는 설명은 환각된 논거가 아니라, 거부를 강제된 수학적 제약의 1:1 직역이다. 생성 모델이 그럴듯한 사유를 지어 내는 자리가 없다. 번역 모델이 자유로울 수 있는 여지는 표현의 매끄러움까지뿐이고, 내용의 빠대는 증명 상태가 고정한다. 불리 조치 통지의 법적 요건이 각 각 어디서 나오는지 추적된다. 구체성과 정확성은 증명 상태에서 나오고, 이해 가능성은 역번역에서 나온다.

이진 판정의 커널과 자연어의 통지문 사이를 이 다리가 잇는다. 커널은 수학의 언어로 단호하게 말하고, 역번역 계층은 그 단호함을 법이 요구하는 말투로 옮긴다. 어느 쪽도 상대의 영역을 침범하지 않는다. 확률을 수학으로부터, 설명을 증명으로 부터 분리해 내려던 이 프로토콜의 기본 그림이 여기서 한 조각 더 맞아 들어간다.

두 개의 출력, 하나의 사실

차단된 행동은 두 개의 출력을 남긴다. 하나는 감사 로그에 적립되는 형식 오류 트레이스고, 다른 하나는 소비자에게 나가는 자연어 통지다. 두 출력은 같은 실패를 서로 다른 언어로 말한다. 여기에 이 구조의 실용적 이득이 있다. 감독관이 통제의 작동을 의심하면 형식 트레이스부터 검사하고, 소비자가 이의를 제기하면 통지문의 사유를 대조하면 된다. 두 기록이 하나의 증명 상태에서 갈라져 나왔기 때문에, 서로 모순될 여지가 구조적으로 없다. 전통적 시스템에서는 통지문을 만든 사람과 차단 규칙을 만든 사람이 달라서 생기던 불일치가 여기서는 번역 계층 하나로 수렴된다.

감사 추적 전체를 넓게 보면 하나의 사슬로 감긴다. 자연어 정책에서 출발해 형식 공리가 되고, 행동의 추측을 거쳐 반증의 오류 상태에 이르며, 다시 소비자의 언어로 돌아온다. 출발점과 끝점이 같은 논리를 공유하기 때문에 중간에 새어 들어갈 해석의 여지가 없다.

다만 다리 위를 건너는 역번역 모델 자체는 여전히 학습된 생성기다. 원문은 이에 대해 별도의 방어를 논하지 않지만, 독자라면 짚어 둘 만하다. 설명의 뼈대를 증명 상태가 고정하기 때문에 환각의 여지가 구조적으로 좁아진다는 것이 원문의 주장이고, 그 주장이 성립하려면 앞 장에서 다룬 개념-심볼 매핑과 같은 격식 장치가 함께 가동돼야 한다. 단단해 보이는 이 구조의 어느 부분이 공격당할 수 있는가. 다음 장이 다룰 질문이 바로 거기다.

DAY

08

공격 표면과 WASM 샌드박스



치명적 삼각형

LLM 에이전트의 보안을 어렵게 만드는 조합이 하나 있다. 도구 접근, 자연어 인터페이스, 신뢰할 수 없는 외부 데이터. 세 가지가 겹치는 순간 에이전트는 외부의 말을 듣고, 그 말을 행동으로 옮길 수단을 진다. 보안 문헌은 이를 ‘치명적 삼각(lethal trifecta)’이라 부른다. 지금까지의 방어는 대체로 경험에 의존했다. 인간 피드백 강화학습(RLHF)과 헌법적 정렬이 모델의 성향을 다듬고, 자동 레드팀이 구멍을 찾아 낸다. 유용하지만 본질적으로 반응적이고 확률적이다. 어제의 공격에 대한 오늘 의 패치일 뿐, 내일의 입력 전체를 보장하지는 못한다.

형식 기법이 이 자리에 들어온다. 안전 필수 하드웨어와 항공우주 공학에서 쓰던 방법으로, 시스템이 특정 기능·보안 속성을 모든 가능한 상태 공간에서 만족함을 보장한다. Lean-Agent Protocol이 여기서 취하는 태도는 냉정하다. LLM은 이미 침해됐다고 가정하라. 모델 내부를 무한히 패치하는 대신, 실행 경계를 수학적으로 검증된 코드로 확보하라는 것이다. 공격을 막는 대신 공격이 통과한 뒤를 설계하는 발상이다.

번역 계층이 공격 표면이다

Lean 커널 자체는 실행 수준에서 절대적이다. 그러나 커널 앞에는 자연어를 Lean 코드로 옮기는 Aristotle 번역 계층이 서 있고, 시스템의 1차 취약점은 바로 여기에 있다. 적이 구조적으로는 온전하지만 의미상 악의적인 Lean 코드를 만들어 내게 만들면, 정책 제약을 통째로 우회할 수 있다. 보안 문헌은 이 위협을 논리적 탈옥과 형식화 드리프트라는 두 이름으로 다룬다.

논리적 탈옥의 대표 격이 LogiBreak(arXiv 2505.13527)와 QueryAttack(ACL 2025)이다. 공격자는 악의적 의도를 복잡한 논리식, 구조화된 비자연 질의어, 암호로 번역한다. 의미는 그대로 유지하면서 프롬프트의 분포를 안전 정렬 학습 데이터의 밖으로 밀어 낸다. 훈련이 본 적 없는 모양의 입력이기 때문에 모델은 요청을 걸러내지 못하고 처리해 버린다.

형식화 드리프트 쪽은 더 은밀하다. 문장 수준 섭동(SLP, arXiv 2506.04575) — 재구술, 무관한 맥락 추가, 동의어 치환 — 이 번역기의 의미 매핑을 흔들어 심볼 드리프트를 낳는다. 간접 프롬프트 인젝션이 제한된 행동 변수를 허용된 Lean 심볼에 매핑시키는 순간, 커널은 자기 역할대로 정확하게 증명에 성공한다. 결과는 정책 위반의 승인이다. 커널은 틀리지 않았는데 시스템 전체가 틀리는, 이 아키텍처 고유의 공격 지점이다. 커널은 형식화된 명제가 증명 가능한지를 검사하지, 형식화 그 자체가 본래 의도를 담았는지는 검사하지 않기 때문이다. 검증의 강도가 강할수록, 번역의 오류는 오히려 더 완벽하게 위장된다.

개념과 심볼을 묶는 표

완화책은 번역 과정에 격식을 강제하는 것이다. MenTaL(Mapping Equivalent Natural language To Assigned Logic)의 접근을 빌려, 번역을 시작하기 전에 개념-심볼 매핑 표를 명시적으로 구축하도록 모델을 몰아붙인다. 다양한 언어 표현을 하드코딩된 Lean 4 심볼 레지스트리에 알고리즘으로 연결하면, 적대적 동의어가 논리 해석기를 납치할 여지가 줄어든다. 자유로운 어휘 선택이 정책의 사본을 만들 기회를 훔쳐 가는 셈이다. 아울러 오케스트레이터는 컨텍스트 수화를 공격적으로 정화한다. 정적 파싱으로 필요한 변수만 뽑아 내고, 신뢰할 수 없는 외부 텍스트는 형식화 엔진의 프롬프트에 닿기 전에 완전히 걸여 낸다. 번역기가 읽는 문서의 어휘 자체를 강판하는 것이다.

에이전트의 코드는 적대적 페이로드다

다음 방어선은 실행이다. 에이전트 AI는 스스로 계획하고, 코드를 쓰고, 실행하도록 설계됐다. 그렇다면 에이전트가 생성한 코드는 원칙적으로 적대적 페이로드로 취급해야 한다. 검증 없이 놓아 두면 원격 코드 실행(RCE), 임의의 파일시스템 조작, 민감 데이터 유출로 직결된다. 신뢰의 기본값이 0이라는 전제가 번역 계층과 실행 계층을 잇는 공통의 출발점이다.

전통의 답인 도커(Docker) 컨테이너에는 구조적 결함이 있다. 컨테이너는 호스트 운영체제의 커널을 공유한다. 에이전트가 만든 페이로드에 커널 익스플로잇이 들어 있으면, 컨테이너 경계를 탈출해 권한을 상승시키고 호스트 머신과 내부 네트워크까지 넘볼 수 있다. 고도로 신뢰할 수 없는 AI 실행을 가두는 용도로는 부족하다는 뜻이다.

수학적으로 갇힌 실행

Lean-Agent Protocol은 성능을 희생하지 않고 총알 같은 격리를 얻기 위해 WebAssembly(WASM)를 실행 기반으로 요구한다. 근사 속도로 코드를 돌리되 수학적으로 안전한 샌드박스 안에 가두는, 극도로 압축된 이식 가능 바이너리 형식이다. 보장은 구조에서 나온다. 첫째, 선형 메모리 모델. 모듈은 지정된 하나의 연속 블록 안에서만 동작하고, 그 밖의 주소에 대한 가시성이 0이라 호스트와 다른 모듈로부터 결함이 엄격히 격리된다. 둘째, 보호 콜스택. 호출 스택이 모듈의 힙 메모리와 논리적으로 분리돼 있어 제어 흐름 하이재킹이 수학적으로 불가능하다. 셋째, WASI(WebAssembly System Interface)의 역량 기반 보안. 최소 권한 원칙에 따라 샌드박스 인스턴스는 호스트의 파일시스템, 네트워크 소켓, 환경변수에 기본 접근 권한이 0이다. 세 보장은 각각 다른 종류의 공격을 막는다. 메모리가 경계를 침범하지 못하게 하고, 스택이 흐름을 납치하지 못하게 하며, 역량이 허락 없는 접촉 자체를 못 하게 한다.

Lean 4 검증 환경과 에이전트의 실행 도구를 모두 WASM 바이너리로 컴파일하면 제로 트러스트 실행 경계가 완성된다. 정교한 적이 Aristotle 번역 계층 탈옥에 성공해 악의적인 Lean 증명을 만들어 냈더라도, 그 결과 실행은 영원히 WASM 샌드박스 안에 갇힌다. 이 장의 구조를 한 장으로 압축하면 아래와 같다. 바깥의 모듈부터 안쪽의 메모리까지, 겹겹의 경계가 실행을 감싸 가둔다.

샌드박스

샌드박스는 네 겹의 경계로 실행을 가둔다

각 계층은 바로 안쪽의 실행을 감싼다.

WASM 모듈 — 에이전트 도구와 검증기

WASI 역량 경계 — 기본 접근 권한 0

보호 콜스택 — 흐름 납치 방지

선형 메모리 — 블록 밖 가시성 0

편집 요약: 본문의 WASM 삼대 보장과 모듈 실행을 계층으로 재배열한 도식이다.

번역 계층의 방어와 실행 계층의 가둠이 서로를 보완하면서, 단일 지점의 실패가 시스템 전체의 침해로 번지는 경로가 끊긴다. 논리적 탈옥이 번역을 속여도 커널이 증명하지 못하면 실행이 없고, 우연히 증명에 성공해도 샌드박스가 피해 반경을 봉쇄한다. 다음 장은 이렇게 완성된 인프라가 시장에서 이미 실제로 굴러가고 있는지를 묻고, 그다음 장에서 기업에 없는 순서를 정리한다. 그러나 그 전에 원문이 제시한 공격 관점 하나를 기억해 둘 가치가 있다. 보안의 질문은 '모델을 얼마나 믿을 수 있는가'가 아니라 '모델을 못 믿는다고 가정한 뒤 무엇을 남기는가'다.

DAY

09

시장과 선례



“증명으로 가드레일을 세우라”는 주장이 설득력을 얻으려면 두 가지를 보여 주어야 한다. 기존 시장의 해법이 왜 부족한지, 그리고 이 방식이 이미 세상 어딘가에서 돌아가고 있는지. 이 장은 원문 논문의 경쟁·시장 분석을 따라 그 두 가지를 확인한다. 경쟁 제품의 기술을 뜯어 보고, 클라우드와 탈중앙 금융의 선례를 지나, 소프트웨어 모호성이 값을 치른 사건들로 마무리한다.

확률의 대표 선수 — NeMo Guardrails

엔비디아의 NeMo Guardrails는 가장 널리 쓰이는 오픈소스 가드레일 툴킷이다. 대화 흐름과 주제 경계를 전용 언어 Colang으로 정의하고, 사용자나 에이전트의 입력이 정한 선 안에 드는지 검사한다.

그 판정 방식이 문제의 씨앗이다. NeMo는 입력이 들어오면 LLM을 호출해 의도의 ‘정규형(canonical form)’을 만들고, 그것을 정해 둔 규칙과 벡터 유사도로 맞춰 본다. 정확한 일치라 아니라 비슷한 정도의 문제로 넘어간 셈이다. 벡터 공간은 연속적이어서, 적대적 섭동 — 단어를 조금씩 바꾸어 넣는 공격 — 이 의미 거리 자체를 조작할 수 있다. 금지된 행동이 ‘비슷해 보이는’ 무해한 의도로 분류되는 빈틈이 수학적으로 열려 있다. 여기에 규칙 판정을 보조하는 LLM-as-a-judge 호출이 더해진다. 판사를 또 하나의 언어 모델로 쓰는 방식인데, 이 호출이 임계 경로에 수백 밀리초의 지연을 엮고 시스템 전체 처리량을 떨어뜨린다. 느린 데다 결정적이지 않은 것이다.

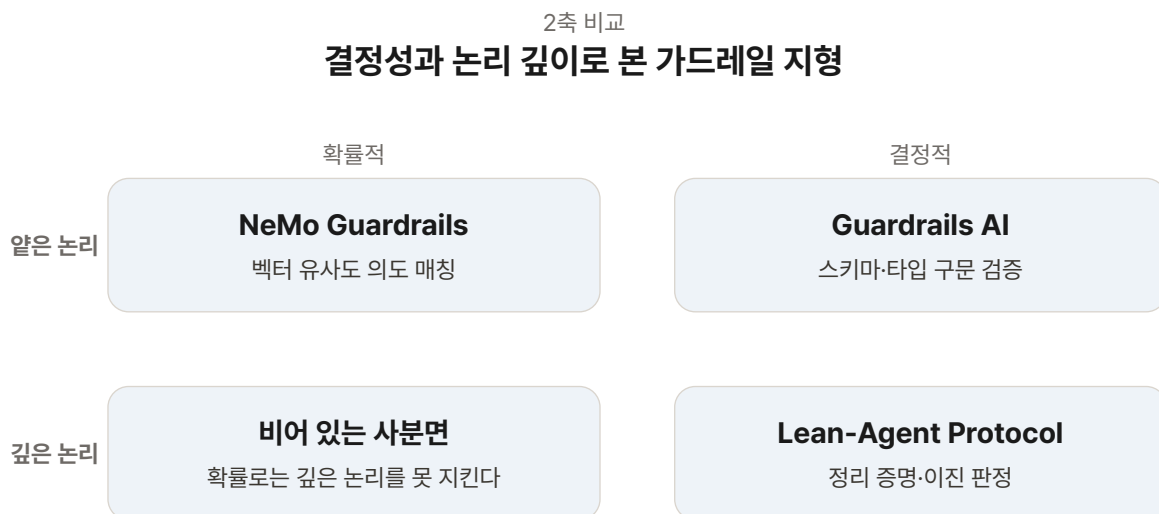
결정적이지만 얇은 — Guardrails AI

가드레일스 AI(Guardrails AI)는 다른 축에 선다. RAIL 명세와 파이썬 기반 라이브러리인 Pydantic으로 출력의 구조적 일관성을 강제한다. 타입이 맞는지, 필드가 있는지, 형식이 틀리지 않았는지를 검사하는 구문 검증이다.

공정하게 말하자. 이 방식은 빠르고 결정적이다. 같은 입력이면 같은 답이 나온다. 그러나 논리적으로 얇다. Pydantic은 에이전트가 제안한 거래량이 정수임을 보장할 수 있다. 그 정수가 과거 마진 사용량과 실시간 자본 제약과 결합했을 때 다층 규제 정책을 지키는지까지 증명하지는 못한다. 구문의 세계와 규제의 세계 사이에는 층이 있다. 결정성을 갖췄으되 논리의 깊이가 한 자 아래에 머무는 것이다.

증명의 축으로 넘어오다

Lean-Agent Protocol은 두 진영의 한계를 각각 다른 방법으로 넘는다. NeMo가 벡터 매칭에 의지한 자리를 폐기하고 의존 타입 언어의 수학적 정리 증명으로 갈아탄다. 확률적 근사가 아니라 증명이니 논리의 깊이가 아예 급이 다르다. 동시에 Guardrails AI의 스키마 검증이 닿지 않는 다변수 제약 조합까지 커버한다. 검사는 마이크로초 단위의 지연으로 이진 확실성을 내놓는다. 아래 도식은 이 세 주자를 결정성과 논리 깊이라는 두 축에 다시 배치한 것이다.



편집 요약: 본문의 세계보 비교를 2축에 재배열한 도식이다.

클라우드의 선례 — AWS Cedar

‘이론은 좋지만 현장에서 돌아간 적이 없다’는 반론은 이미 실증적으로 반박된다. 가장 유력한 근거가 아마존웹서비스의 Cedar다. 수십억 규모 클라우드의 권한 부여 정책 언어인 Cedar는 검증 주도 개발(verification-guided development)이라는 방법론으로 만들어졌다. 평가기와 권한부여기, 검증기라는 핵심 구성요소를 Lean 4로 형식 모델링하고 그 위에서 정리를 증명한 뒤 구현을 쌓았다.

증명된 정리의 예가 인상적이다. “금지(forbid) 정책은 허가(permit) 정책보다 평가 순서와 무관하게 항상 우선한다”는 명제가 Lean 4로 증명되어 있다. 정책이 어떤 순서로 섞여 평가되더라도 금지가 이긴다는 보장이 수학적으로 담혀 있는 것이다. AWS는 이 구조로 천조(퀴드рили언) 단위의 프로덕션 권한 부여를 처리해 왔고, 형식화된 Lean 모델의 평가는 앞서 본 대로 평균 5마이크로초였다. 최고 처리량의 기업 인프라가 형식 검증을 견디지 못한다는 통념은 실제 배치 뒤에서 무너져 있다.

탈중앙 거래소의 선례 — AMM의 형식화

금융공학의 다른 선례가 탈중앙 금융(DeFi)에서 온다. 유니스왑류 자동화 시장 조성자(AMM)는 상수곱 공식 $x \cdot y = k$ 로 두 자산의 풀 가격을 결정하고, 가격이 어긋나면 차익거래가 스스로 맞추며, 수수료가 유동성 공급자에게 돌아간다. 이 경제적 안전성은 지금까지 종이와 연필의 증명에 의존했다. 사람이 쓴 수학 논문이 보증하는 방식이다.

연구자들은 이 속성들을 Lean 4 정리증명기로 직접 형식화했다. 상수곱 공식과 차익거래 기회, 복잡한 수수료 메커니즘까지 기계 검증 가능한 정리로 옮긴 것이다(FMBC 2024 발표 연구, 이후 arXiv 2602.00101의 수수료 메커니즘 확장까지 이어졌다). 종이-연필 증명에는 사람의 눈이 빠뜨린 단계가 남을 수 있지만, 기계 검증은 그 빈틈을 허용하지 않는다. 경제 메커니즘의 보증 주체가 ‘사람의 검토’에서 ‘커널의 판정’으로 옮겨 간 실제 사례다.

소프트웨어 모호성의 대가

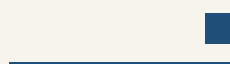
왜 금융은 이렇게까지 엄격해야 하는가. 역사가 답을 갖고 있다. 논문이 드는 두 사건을 보자. 먼저 나이트 캐피털 사건(2012). 통제되지 않은 알고리즘 오류가 45분 만에 4억 4천만 달러의 손실을 냈고, 회사는 존립 자체가 흔들렸다. 다음은 2007년의 ‘해머’ 사건. 기초적인 알고리즘이 조작되어 뉴욕상품거래소(NYMEX) 선물의 결제 가격이 불법적으로 움직였다. 정의상 결제 가격은 시장 전체가 신뢰하는 기준점인데, 그 기준점이 코드의 모호한 틈으로부터 흔들린 것이다.

두 사건의 공통분모는 명확하다. 소프트웨어의 모호성 — 정의되지 않은 경로, 검증되지 않은 조건 — 이 자본과 시장 신뢰로 전환된 점이다. 금융 시장은 소프트웨어 모호성을 용인하지 않는다. 확률적 에이전트의 시대에 이 문장은 더 무거워진다. 모호성이 이제 사람의 실수뿐 아니라 모델의 ‘그럴듯함’에서도 샌다. 그래서 논문의 결론이 향하는 곳은 하나다. 통계적으로 아마 맞을 행동이 아니라, 증명할 수 있는 행동만 시장에 내보내는 것. 시장이 이미 치른 대가가 그 방향의 근거다.

DAY

10

전자정부프레임워크의 가드레일



"원문: Position: AI Governance Needs ISO-like Interoperability Protocols, Not Just Laws — arXiv 2608.14568(2026) · Azmine Touseh Wasi 외 5인"

"링크: <https://arxiv.org/abs/2608.14568>"

"원문: Digital Transformation in the Public Administrations: a Guided Tour For Computer Scientists — arXiv 2305.05551(2023) · Paolo Ciancarini·Raffaele Giancarlo·Gennaro Grimaudo"

"링크: <https://arxiv.org/abs/2305.05551>"

표준이라는 이름의 울타리

한국 행정 정보화의 밑그림에는 표준프레임워크, 흔히 전자정부프레임워크라 부르는 공통 개발·실행 환경이 깔려 있다. 공공기관이 업무 시스템을 지을 때 인증, 권한, 파일 처리, 화면 구성 같은 기능을 부품처럼 꺼내 쓰게 하는 공통컴포넌트가 그 심장이다. 각 기관이 같은 기능을 제각기 만들면 품질이 갈라지고, 검증도 감사도 일마다 따로 해야 한다. 그래서 '한 번 검증된 부품을 다 함께 쓴다'는 원칙이 세워졌고, 개발자는 프레임워크가 정한 규격 밖으로 나가기 어렵게 되었다.

이 구조를 이 책의 언어로 바꿔 보면 놀랍게 익숙하다. 공통컴포넌트는 미리 검증된 코드의 정책 환경이다. 개발표준을 강제하는 일은 규정을 미리 컴파일해 두는 일이다. 물론 이것은 필자의 각색이다. 원문 논문은 금융 규제와 Lean 4를 다룰 뿐, 한국 행정을 이야기하지 않는다. 그러나 '공통 검증 계층을 관문에 세우고 우회로를 막는다'는 설계는, AI 에이전트의 모든 행동을 증명 게이트 앞에 세우는 아이디어와 같은 뼈대를 공유한다. 차이는 검증의 깊이다. 프레임워크가 코드가 돌아가는지를 검사한다면, 증명 게이트는 코드가 규정을 지키는지를 검사한다.

법보다 프로토콜

첫 번째 논문은 AI 거버넌스의 방향을 놓고 논쟁하는 입장 논문이다. 저자들은 EU의 AI법, 중국의 알고리즘 거버넌스, 미국의 위험관리 프레임워크처럼 나라마다 갈라져 나오는 법과 자율 규범이 오히려 조각난 규제 풍경을 만들고 있다고 진단한다. 대안으로 내놓는 것은 국제표준화기구(ISO)류의 상호운용성 프로토콜이다. 편향, 에너지 사용량, 데이터 출처를 통일된 지표로 담아 기계가 읽을 수 있게 만든 AI 영양성표를 표준으로 삼자는 것이다. 규정 준수의 무게중심을 나라마다 다른 '법적 합치'에서, 어디서나 판정 가능한 '기술적 합치'로 옮기자는 주장이다.

한국 행정의 표준·인터페이스 체계와 겹쳐 보면 논점이 또렷해진다. 행정 정보시스템끼리 연계할 때는 표준화된 인터페이스와 데이터 형식을 따른다. 법 집행이 아니라 프로토콜 준수가 연계를 가능하게 한다는 사실은 이미 우리가 경험한 것이다. 이 논문의 통찰을 한 문장으로 좁히면 이렇다. 판정을 자동으로 하려면 규칙이 기계가 읽을 수 있는 형태여야 한다. Lean-Agent의 정책 환경이 정확히 그 지점에 서 있다. 법 조문을 통째로 담지 않고, 법이 요구하는 조건을 기계가 참과 거짓을 판별할 수 있는 명제로 바꾸어 담는다. 상호운용성 논문과 증명 게이트는 같은 질문의 두 답이다. 하나는 서로 다른 조직이 같은 말을 하게 하는 것이고, 하나는 같은 말을 기계가 검사하게 하는 것이다.

공공 디지털 전환의 네 기둥

두 번째 논문은 공공 행정의 디지털 전환을 컴퓨터 과학자에게 안내하는 강의문이다. 문헌 조사를 바탕으로 성공한 전환이 네 개의 기둥 위에 선다고 정리한다. 공개 데이터, 정보통신기술, 시민과 공무원의 디지털 역량, 그리고 새 디지털 서비스를 만드는 애자일 프로세스다. 저자들은 데이터를 첫 기둥으로 꼽으면서, 기술 도입만으로는 전환이 아니라는 점을 거듭 강조한다. 조직의 문화와 관계 방식이 함께 바뀌어야 한다는 것이다.

한국 전자정부는 이 진행의 앞자리에 있다. 전자정부법을 일찍 제정한 나라로 꼽히고, 민원을 온라인 창구로 내는 인프라도 일상 수준에 자리 잡았다. 과장 없이 말해도 공공 서비스의 디지털 전환은 국가 단위 실천이 되었다. 그러나 네 기동 논문이 경고하듯, 전환은 언제나 다음 질문을 낳는다. 창구의 주인공이 바뀌면 조직은 무엇을 바꿔야 하는가. 지금 그 질문의 답이 기계라는 예감이 들고 있다.

민원 창구에 에이전트가 앉았다면

이제 상상해 보자. AI 에이전트가 민원 접수를 읽고, 유형을 판별하고, 내부 시스템을 조회해 답을 쓰고 통보까지 마친다. 이 장면을 원본 논문의 언어로 번역하면 그림이 선명해진다. 에이전트가 세운 처리 계획은 하나의 추측이다. 실행하기 전에 증명 게이트에 올려 검사받는다. 검사 기준이 되는 정책 환경의 행정 버전은 법령과 행정규칙, 업무 지침의 형식화다. 예컨대 “개인정보 열람은 해당 사건의 범위 안에서만”, “처리 기한은 접수일로부터 계산한다” 같은 조항이 명제가 된다. 이것 역시 필자의 각색이다. 원문은 이런 행정 장면을 다루지 않는다.

구체적인 한 판을 굴러 보자. 에이전트가 “이 민원은 이번 주 내에 답변 통보”라는 계획을 세웠다. 게이트는 계획을 명제로 바꾸어 물었다. 담당자의 열람 권한 범위가 사건과 일치하는가. 처리 기한 산식이 지침과 같은가. 통보문의 개인정보 필드 목록이 규정 허용 목록 안에 있는가. 셋이 모두 증명되면 실행하고, 하나라도 안 되면 막는다. 막혔다면 어디서 막았는지가 기록으로 남는다. 여기서 중요한 점은 에이전트가 똑똑하느냐가 아니라, 통과 여부를 결정하는 것이 수학이라는 사실이다.

여기서 눈길을 끄는 것은 결재 흐름이다. 기안에서 합의를 거쳐 결재에 이르는 절차는 원본 논문의 승인 루프와 동형이다. 에이전트의 제안이 기안이라면, 증명 게이트는 기계가 담당하는 합의이고, 결정적인 사안에서 사람의 승인은 마지막 결재다. 증명 게이트가 무엇을 보장하는지 따져 보면 이렇다. 에이전트가 처리를 끝냈다고 말하기 전에, 기한 제약과 권한 범위, 정보 열람 한계를 모두 만족함이 기계적으로 확인된다. 하나라도 증명이 안 되면 행동은 차단되고, 어느 조항에서 막혔는지에 대한 감사 증거가 남는다. 확률적 필터가 ‘그럴듯한 처리’를 보여 주는 것과 결정적인 차이이다.

행정 시스템의 개념	Lean-Agent의 개념	의미
법령·행정규칙·업무 지침	정책 환경	기계가 판정할 명제의 모음
기안→합의→결재	승인 루프	행동 실행 전의 검사 관문
시스템 연계 표준	인터페이스 계약	우회로를 막는 접점
감사 증거	오류 추적	차단 사유의 기록

표준에서 증명까지의 사다리

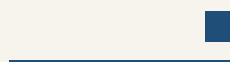
한계도 분명하다. 전자정부프레임워크의 검증은 실행 의미에 가깝다. 부품이 정해진 환경에서 망가지지 않고 돌아가는지를 다룬다. 반면 증명 게이트가 요구하는 것은 의미론이다. 이 코드가 이 규정을 지킨다는 명제의 증명이다. 두 계층 사이에는 아직 거리가 있다. 법령의 자연어 문장을 형식 명제로 옮기는 일은 전자정부프레임워크의 사업 영역 밖이다.

그래서 현실적인 길은 사다리다. 첫 칸은 데이터 표준이다. 이미 갖추어진 표준 형식 위에 둘째 칸인 API 계약을 놓는다. 어떤 요청이 어떤 조건에서만 허용되는지 계약 수준에서 명문화한다. 셋째 칸은 속성 검증이다. 계약이 보장한다고 선언한 속성을 기계가 판정하는 단계로, 민원 처리 규칙 가운데 기한이나 권한처럼 결정론적인 것부터 시작한다. 오늘의 행정 조직이 할 수 있는 첫걸음은 이 셋째 칸에 있다. 사람이 해석해야만 하던 규칙을 골라내고, 기계가 판정할 수 있는 문장으로 바꾸어 두는 일. 그러면 에이전트가 창구에 앉는 날, 증명 게이트는 새로 짓는 것이 아니라 이미 쌓아 둔 계단을 오르지만 하면 된다.

DAY

11

CMS·PMS·Slack의 가드레일



“원문: Enabling Content Management Systems as an Information Source in Model-driven Projects — arXiv 2508.19797(2025) · Joan Giner-Miguel·Abel Gómez·Jordi Cabot”

“링크: <https://arxiv.org/abs/2508.19797>”

“원문: From Chatbot to Digital Colleague: The Paradigm Shift Toward Persistent Autonomous AI — arXiv 2606.14502(2026) · Yongheng Zhang 외 19인”

“링크: <https://arxiv.org/abs/2606.14502>”

누구나 쓰는 도구, 누구나 통과하는 관문

콘텐츠 관리 시스템(CMS), 프로젝트 관리 도구(PMS), 업무용 메신저는 기업 IT의 바닥층이다. 몇 명 규모의 팀도, 수천 명 조직도 이 도구들을 매일 쓴다. 금융 거래처럼 드라마틱한 사고가 나지 않아서 가드레일 논의에서 뒷전이 되기 쉽지만, 다루는 문제는 결코 가볍지 않다. 발행되는 콘텐츠의 무결성, 승인 절차의 준수, 권한의 경계. 이 세 가지는 원본 논문이 금융에서 행간 사전거래 통제, 승인 루프, 감사와 정확히 대응한다. 도구가 일상적일수록 통제가 투박해지기 쉽고, 그래서 결정론적 원칙을 옮겨 볼 가치가 오히려 크다.

콘텐츠 모델이 곧 타입 계약이다

첫 번째 논문은 헤드리스 CMS를 다룬다. 최근의 CMS는 사람이 브라우저로 볼 화면을 만드는 도구에서 벗어나, REST API를 통해 다른 애플리케이션이 소비할 콘텐츠를 공급하는 정보원으로 진화했다. 문제는 콘텐츠의 구조가 기관마다 제각기 꾸려져 있다는 점이다. 저자들은 CMS 안쪽의 정보 스키마를 자동으로 발견해 명시적인 모델로 표현하고, 그 모델과 클라이언트 사이의 상호작용을 미들웨어로 생성하는 모델 기반 프레임워크를 제안한다.

이 제안의 밑바닥에는 하나의 신념이 있다. 콘텐츠의 구조를 발견하고 명시하면, 그 구조가 계약이 된다. 필자는 여기에 증명 게이트의 콘텐츠 버전을 겹쳐 본다. 발행 버튼 앞에 검증 관문을 세우는 것이다. 필수 필드가 채워졌는지, 참조하는 카테고리 이미지와 이미지가 실제로 존재하는지, 콘텐츠 모델의 제약을 어겼는지. 이 검증은 속도가 빠르고 결과가 결정적이다. 스키마 검증만으로는 논리적으로 알다는 2장의 비판도 적용되지만, 발행 무결성이라는 좁은 문제에서는 결정론적 관문이 이미 실전에서 쓰이고 있다는 점이 중요하다. 금융의 사전거래 통제가 주문을 시장에 보내기 전에 걸러낸다면, CMS 발행 게이트는 문서를 세상에 보이기 전에 걸러낸다.

챗봇에서 디지털 동료로

두 번째 논문은 방대한 서버로 대형 언어 모델의 진화를 정리한다. 대화 답변기였던 챗봇이 추론을 길게 펼치는 사고형 모델로, 그리고 도구를 부르는 에이전트를 넘어 지속적인 작업 공간과 재사용 가능한 스킬을 갖춘 ‘디지털 동료’로 가는 경로를 그린다. 핵심 개념은 작업 공간과 스킬의 결합이다. 파일과 로그, 권한이 남는 작업 공간에 상태가 쌓이고, 검증 절차가 붙어서 한 번의 대답으로 끝나던 일이 완결된 작업으로 바뀐다.

서베이가 짚는 평가의 전환도 인상적이다. 답이 그럴듯한지를 묻던 평가에서 벗어나, 작업이 끝난 뒤의 상태가 의도에 부합하는지, 그 과정이 재현 가능하고 감사 가능한지를 묻는 과제 완결 평가로 옮겨 간다는 것이다. 채팅 하나의 성적표가 아니라 일감 하나의 성적표다. 이 전환의 저변에는 단순한 신념이 깔려 있다. 검증 가능한 기준이 없으면 신뢰도 만들 수 없다는 것.

이 근미래에서 Slack류 협업 공간은 에이전트의 근무지가 된다. 채널에서 지시를 받고, 티켓을 만들고, 배포를 돌리는 챗웁스가 이미 있다. 대화로 운영 명령을 내리는 방식은 속도를 준다. 그러나 속도의 그늘에서 위험이 커진다. 자연어 지시는 모

호하고, 채널을 지나가는 누군가의 한 마디가 명령으로 오해될 수 있다. 서버이가 지적하듯, 출력이 실행 가능한 행동이 되는 순간 안전 문제는 언어의 문제에서 실행 경계의 문제로 바뀐다. 정책은 행동이 현실이 되는 자리에서 강제되어야 한다는 말이다. 증명 게이트의 철학과 정확히 같은 결론이다.

결재는 오케스트레이터다

여기서부터는 필자의 각색이다. 프로젝트 관리 도구의 티켓 승인, 그리고 회사의 기안 결재 흐름은 승인 루프의 기업 일상 버전이다. 에이전트가 티켓 담당자를 정하고 일정을 배치하려 할 때, 그 제안을 하나의 추측으로 취급하면 이야기가 단순해진다. 정책 환경에는 “담당자는 해당 영역의 멤버여야 한다”, “마감 임박 티켓은 사람의 확인을 받는다” 같은 명제가 담겨 있고, 증명이 되면 자동으로 넘어가고 안 되면 사람의 결재로 보내진다.

Slack 봇의 설계 관습을 펼쳐 보면 오케스트레이터의 축소판이 이미 흩어져 있다. 민감한 명령에는 명시적인 승인 버튼을 붙이는 관습, 봇에게 채널별로 권한 범위를 좁혀 주는 관습, 되돌릴 수 없는 행동 앞에 사람을 세우는 관습. 이 세 가지는 각각 행동 차단, 샌드박스의 권한 최소화, 인간 개입 지점에 해당한다. 결정적인 차이는 판정의 주체다. 오늘의 관습은 개발자의 경험과 제품의 옵션에 맡겨져 있지만, 증명 게이트는 이 경계를 수학적 명제로 고정한다. ‘되도록’이 아니라 ‘증명된’ 승인 절차가 되는 것이다.

디자인 시스템은 시각 언어의 정책 환경이다

웹디자인도 예외가 아니다. 디자인 토큰은 색, 간격, 타이포그래피 값을 이름 붙은 불변의 집합으로 묶어 두는 방법이고, 업계에서는 이를 교환 가능한 형식으로 표준화하는 흐름이 진행 중이다. 토큰은 정책 환경과 닮았다. ‘여기서는 이 색만 쓴다, 이 간격만 쓴다’는 규칙이 코드화되어 있고, 컴포넌트는 그 규칙 안에서만 조립된다. 접근성 대비 검증은 결정론적 규칙의 좋은 예이기도 하다. 전경색과 배경색의 대비비가 기준을 넘는지는 산술로 판정된다. 확률이 아니라 계산이다.

이렇게 보면 디자인 시스템은 시각 언어의 정책 환경이고, 린터와 자동 대비 검사는 축소된 증명 게이트다. 마지막 고개(14장)의 결론을 미리 연습하는 셈이다. 타입 검사가 축소된 증명이라면, 대비 검사는 시각 언어의 축소된 증명이다. 아래 그림은 이 장의 축을 하나로 압축한다. 콘텐츠 발행이 어떤 관문을 통과하는지, 검증에 실패하면 어디로 되돌아가는지를 그려 본 것이다.

발행은 게이트를 통과한 콘텐츠만 출구가 있다

CMS 발행 승인 파이프라인 — 검증 실패는 초안으로 되돌아간다

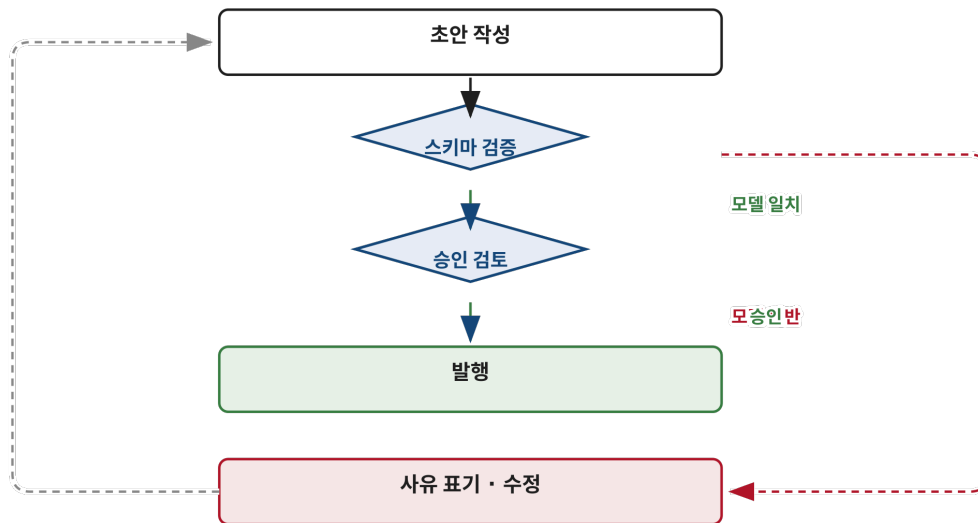


그림 11-1 콘텐츠 발행 승인 게이트 — 스키마 검증과 승인 검토의 이중 관문

자연어 명령의 애매함을 좁히는 실용적인 방법도 있다. 봇이 받은 명령을 행동 후보로 바꾸되, 실행 전에 그 후보를 정해진 양식으로 요약해 승인자에게 보여 주는 것이다. “이 저장소를 배포하고, 채널에 결과를 올립니다”라는 한 줄 요약은 열 장의 대화보다 검사하기 쉽다. 요약 양식이 티켓 양식처럼 표준화되면, 그 요약 자체가 스키마 검증의 대상이 된다. 대화의 흐름을 명제의 흐름으로 바꾸는 첫 단추다.

일상 도구에서 배우는 것

이 장의 세 장면 — 발행 게이트, 챗봇의 실행 경계, 승인 버튼과 권한 범위 — 은 각자 따로 자라온 관습처럼 보인다. 그러나 한데 놓으면 하나의 원칙이 보인다. 사람의 주의를 기대지 말고, 관문에 검증을 심어라. 일상 도구는 규제의 사각지대처럼 보이지만, 오히려 누구나 접하는 만큼 원칙이 가장 널리 퍼질 자리다. 민감한 배포 명령 앞에 승인을 요구하는 봇 하나가 조직의 안전 문화를 가르친다. 오늘의 관습을 명제로 다듬는 일, 그것이 CMS와 Slack의 세계에서 증명 게이트를 준비하는 방법이다.

DAY

12

UI를 짓는 언어들의 가드레일 — Flutter·React Native·TypeScript



“원문: From Logic to Toolchains: An Empirical Study of Bugs in the TypeScript Ecosystem — arXiv 2601.21186(2026) · TianYi Tang·Saba Alimadadi·Nick Sumner(사이먼 프레이저 대학교)”

“링크: <https://arxiv.org/html/2601.21186v1>”

“원문: MobileDev-Bench: A Comprehensive Benchmark for Evaluating Language Models on Mobile Application Development — arXiv 2603.24946(2026) · Moshood A. Fakorede·Krishna Upadhyay·A. B. Siddique·Umar Farooq(루이지애나 주립대학교·켄터키 대학교)”

“링크: <https://arxiv.org/html/2603.24946v1>”

“원문: Developing Accessible Mobile Applications with Cross-Platform Development Frameworks — arXiv 2005.06875(2020) · Sergio Mascetti·Mattia Ducci·Niccolò Cantù·Paolo Pecis·Dragan Ahmetovic(밀라노 대학교)”

“링크: <https://arxiv.org/html/2005.06875v1>”

타입 검사는 축소된 증명이다

원본 논문의 주장을 한 문장으로 줄이면 이렇다. 규정을 기계가 검사할 수 있는 명제로 만들고, 증명되지 않은 행동은 실행하지 마라. 이 장은 그 원리를 우리가 매일 쓰는 도구로 내려 옮긴다. 화면을 짓는 언어들 — TypeScript, Flutter의 Dart, React Native — 이 그 대상이다. 2부의 다른 장과 마찬가지로, 원본 논문의 개념을 UI 계층에 대입하는 일은 필자의 각색이다. 원본 논문은 금융 규제와 Lean 4만을 다룬다. 다만 각 절의 사실관계가 되는 논문들은 장 머리에 밝힌 대로 실물을 확인해 읽었다.

들어가며에서 예고한 문장이 있다. 타입 검사가 축소된 증명이라면, 증명은 완성된 타입 검사다. UI 계층은 이 사다리의 아래쪽 칸에 선다. 그래서 오히려 보기 좋다. 가이드릴이 어떻게 만들어지고, 어디까지 막아 주고, 어디서부터 통제를 놓치는지를 가장 선명하게 드러내는 곳이기 때문이다.

633개의 버그가 말해 주는 것

TypeScript가 정말 버그를 줄이는지 묻는 대규모 실증은 놀랍게도 이 논문이 처음이다. 연구진은 16개 공개 저장소에서 633개의 버그 보고를 수집하고 하나하나 분류했다. 결과의 어조가 흥미롭다. 정적 타입은 확실히 일을 한다. 런타임 예외는 전체의 1.2퍼센트, 오류 처리 결함은 0.8퍼센트에 그쳤다. 자바스크립트 시절 연구와 비교하면 타입 관련 결함은 약 33퍼센트에서 12.4퍼센트로 떨어졌고, 과거 클라이언트 자바스크립트 버그의 68퍼센트를 차지하던 DOM 직접 조작 문제는 프레임워크가 DOM을 대신 다루면서 크게 물러났다.

그러면 무엇이 버그의 주범이 되었는가. 도구와 설정, 그리고 경계면이었다. 빌드·컴파일러·설정 결함이 27.8퍼센트로 가장 많았고, API 오용이 14.5퍼센트, UI 버그가 12.5퍼센트가 뒤를 이었다. `tsconfig.json` 한 줄의 경로 오류나 번들러 설정의 불일치처럼, 코드 밖의 계층에서 태어나 조용히 파이프라인을 타고 퍼지는 결함이다. 연구진은 이를 한마디로 압축한다. 신뢰성의 병목이 코드 안의 논리에서 생태계의 조율로 이동했다는 것이다.

여기에 가이드릴 사고의 기본 자세가 들어 있다. 도구가 무엇을 막아 주는지, 그리고 무엇을 막지 못하는지를 데이터로 확인하는 일이다. 타입 시스템의 보장은 그 시스템이 검사하는 명제 안에서만 유효하다. 보장의 경계를 알 때 우리는 다음 계단을 어디에 놓을지 판단할 수 있다. 이것은 원본 논문이 기존 AI 가이드릴의 한계를 짚던 방식과 정확히 같은 자세다.

언어 안에 내장된 가드레일

Dart의 사운드 null 안전성은 언어 계층 가드레일의 표본이다. 널이 될 수 있는 값과 없는 값을 타입 자체로 구분하고, 널 참조 가능성을 컴파일 시점에 원천 차단한다. 개발자가 런타임 검사를 성실하게 하기로 다짐하는 대신, 컴파일러가 대신 집행한다. 필자의 각색을 미리 밝히는데, 이것은 미니 정리라 부를 만하다. `String?` 타입은 “이 값은 없을 수 있다”는 명제고, 컴파일러는 그 명제를 다루는 코드마다 널이 아님을 증명할 것을 요구한다. 증명이 안 되면 컴파일이 막는다. Lean 커널이 증명에 실패한 행동의 실행을 막는 것과 같은 이진 구조다. 규칙이 문서에 있느냐, 집행기 안에 있느냐의 차이가 두 세계를 잇는 축이다.

그렇다면 AI가 이 언어들로 코드를 쓰는 시대에는 어떤가. MobileDev-Bench가 그 질문에 숫자로 답한다. 이 벤치마크는 실제 서비스 중인 모바일 앱 18개에서 실제 이슈 해결 과제 384개를 수집했다. 안드로이드 네이티브 307개, Flutter 63개, React Native 14개다. 네 개의 최신 코드 모델을 풀어 보니 종합 해결률이 3.39퍼센트에서 5.21퍼센트 사이에 불과했다. 한 자릿수의 한가운데다, 어려운 과제에서는 0퍼센트에서 1.6퍼센트로 사실상 무너졌다. 원인도 명확하다. 모바일 앱의 수정은 평균 12.5개 파일, 325줄에 이르고, 소스코드 외에 설정·리소스·매니페스트 같은 서로 다른 종류의 파일을 함께 건드려야 한다. 모델은 고쳐야 할 파일을 찾는 단계에서 이미 무너졌다. 파일 수준 재현율이 13.8퍼센트에서 19.5퍼센트 사이였다.

원본 논문의 승인 루프를 떠올려 보자. 에이전트의 행동을 중간에서 가로채 증명을 요구하고, 통과하면 실행한다. 해결률이 5퍼센트인 생성기가 있다면, 문제는 생성기의 능력만이 아니라 그 출력을 어디서 걸러내는가다. 빌드가 곧 검증이고 테스트 통과가 곧 증명인 Flutter·React Native 프로젝트에서, 이 게이트를 지우면 AI가 쓴 코드가 곧장 사용자의 화면으로 간다. UI 코드를 기계가 쓰는 시대에 검증 게이트는 사치가 아니라 전제다.

결정론적으로 검사할 수 있는 UI 규제

접근성은 UI 규제가 결정론적 검사와 만나는 지점이다. 밀라노 대학교 연구진은 iOS와 안드로이드의 스크린리더 API를 체계적으로 분석해 25개의 기능을 목록으로 만들었다. 그중 14개는 두 플랫폼에 기능적으로 동등하게 존재했다. 이론상 크로스플랫폼 프레임워크는 14개를 모두 하나의 API로 감쌀 수 있었다. 실제로는 React Native가 8개, Xamarin이 5개를 감쌌을 뿐이다. 가장 기본적인 8개 기능조차 React Native는 6개, Xamarin은 5개만 노출했다. 감싸지 않은 API를 쓰려면 개발자가 네이티브 언어로 플랫폼별 코드를 따로 작성해야 한다. 연구진의 실험에서는 이런 우회 구현에 며칠에서 수주가 걸렸고, 결국 크로스플랫폼의 이점을 갉아먹었다. 추상화가 규제 준수를 누수시키는 사례다.

접근성 규칙의 성질에 주목하자. 이미지에 대체 텍스트가 있는지, 포커스 순서가 논리적인지, 레이블이 붙었는지 — 이런 규칙은 해석의 여지가 좁고 기계가 판정할 수 있다. 웹 접근성 표준의 자동 검증 도구가 그 증거다. 자연어로 쓰인 규제를 기계가 판정하는 형태로 옮기는 작업, 원본 논문이 자동 형식화라 부른 그 일의 UI판인 셈이다. 규모는 작아도 구조는 같다.

UI 계층의 가드레일 사다리

이 장의 재료를 한 계단식으로 세워 보자. 린트가 흔한 실수를 걸러내고, 타입 검사가 명제를 강제하고, 빌드와 테스트가 합쳐지지 않은 코드를 막고, 디자인 토큰이 색과 간격의 값을 단일 출처로 묶고, 접근성 검사기가 규제 준수를 점점한다. 각 계단은 위로 갈수록 더 깊은 성질을, 더 결정론적으로 검사한다. 다음 표는 이 대응을 정리한 것이다. 이 표와 아래 해석은 필자의 각색이다.

UI 계층의 도구	하는 일	원본 논문의 개념
린터	흔한 실수·스타일 위반을 걸러냄	구문 수준 검증
타입 검사기	타입 명제를 컴파일 시점에 강제	축소된 증명
Dart의 사운드 null 안전성	널 참조를 원천 차단하는 이진 판정	커널의 허가·차단
빌드·테스트 게이트	증명 없는 병합을 막음	승인 루프
디자인 토큰	값을 불변의 단일 출처에 둬	정책 환경

UI 계층의 도구	하는 일	원본 논문의 개념
접근성 검사기	검사 가능한 UI 규제를 자동 판정	자동 형식화된 규제

사다리를 다 오르면 어디에 닿는가. 원본 논문이 놓인 자리다. 이 장의 도구들은 규칙을 코드와 설정에 심지만, 그 규칙은 아직 언어가 정의한 명제 안에 머문다. 원본 논문은 사업의 규정 자체를 증명의 대상으로 만든다. 타입 검사가 축소된 증명이라는 문장은, 반대로 읽으면 증명까지 오르는 길이 이미 우리 발밑에 시작되어 있다는 뜻이다.

DAY

13

시스템을 짓는 언어들의 가드레일 — Rust·Go·Python



"원문: Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs — arXiv 2003.03296(2020) · Hui Xu·Zhuangbin Chen·Mingshen Sun·Yangfan Zhou·Michael R. Lyu(푸단대학교·홍콩중문대학교·바이두 보안)"

"링크: <https://arxiv.org/html/2003.03296>"

"원문: A Study of Real-World Data Races in Golang — arXiv 2204.00764(2022) · Milind Chabbi-Murali Krishna Ramanathan(Uber)"

"링크: <https://arxiv.org/abs/2204.00764>"

"원문: Getting Python Types Right with RightTyper — arXiv 2507.16051(2025) · Juan Altmayer Pizzorno·Emery D. Berger(매사추세츠 대학교 애머스트)"

"링크: <https://arxiv.org/pdf/2507.16051>"

언어가 곧 규제다

원본 논문은 사실상 하나의 신뢰 위에 서 있다. Lean 커널이 정직하다는 신뢰다. 커널이 작고, 검사 가능하고, 거짓 증명을 통과시키지 않기 때문에 증명의 판정을 맡길 수 있다는 논리다. 이 장은 그 구조를 시스템 언어에 비취 본다. 우리가 이미 소 유권과 타입을 신뢰하는 것과 같은 구조다. 언어의 타입 규칙은 권고문이 아니라 집행이다. 규칙을 어기면 경고가 아니라 컴파일 실패가 온다. 2부의 다른 장과 마찬가지로 언어 계층에 원본의 개념을 대입하는 일은 필자의 각색이며, 각 절의 사실 관계는 장 머리의 논문에서 확인했다.

세 언어는 이 집행의 강도가 저마다 다르다. Rust는 집행을 벗어나는 통로를 따로 만들어 두었고, Go는 중요한 영역을 관례에 맡겼고, Python은 계약서를 쓰게는 하되 제출을 강제하지 않는다. 강제와 관례 사이의 이 세 지점을 차례로 본 뒤, 원본 논문이 있는 수학 계층과의 거리를 재보자.

safe의 약속, unsafe의 경계

Rust는 메모리 안전성을 언어의 약속으로 내걸었다. 그 약속이 깨졌는지 묻는 가장 정직한 방법은 실제 사고 기록을 전수 조사하는 것이다. 연구진은 2020년 말까지 보고된 모든 메모리 안전성 관련 CVE를 포함해 실제 버그 186건을 모아 분석했다. 결론은 명확했다. 컴파일러 자체의 버그 한 건을 빼면 나머지 전부가 unsafe 코드를 필요로 했다. Rust의 약속은 유지되고 있었다. 사고의 양상도 읽을 만하다. 결과 유형으로는 사용 후 해제가 82건으로 가장 많았고, 원인 유형으로는 소유권 규칙과 상호작용하며 터지는 불완전한 제네릭·트레이트가 52건, 자동 메모리 회수 문제가 35건, 안전하지 않은 함수가 29건이었다. 연구진은 이 다수가 즉시 침해를 일으키지는 않지만 unsafe 없이도 버그를 쓸 여지를 남기는, 경미한 건전성 문제임을 짚었다.

여기서 이 장의 핵심 유비를 꺼낸다. 필자의 각색임을 밝혀 둔다. unsafe 블록은 원본 논문 8장의 번역 계층 취약점과 동형이다. 원본 논문은 Lean 세계의 최약점이 실행이 아니라 번역에 있다고 경고한다. Aristotle이 정책을 오역한 Lean 코드를 만들어도, 그 코드가 구조적으로 유효하면 커널은 정직하게 증명을 통과시킨다. 형식화의 표류가 바로 그 공격이다. Rust에서 unsafe 마커 안의 코드가 정확히 같은 자리다. 컴파일러는 그 구간에서 검사를 포기하고, 개발자의 판단을 그대로 믿는다. 경계 밖의 보장은 아무리 단단해도, 경계 안의 오류는 그대로 통과한다. 그래서 원본 논문이 번역 계층을 개념-심볼 매핑 테이블과 컨텍스트 살균으로 잠그는 처방을 내리는 것처럼, Rust 생태계의 처방은 unsafe를 최소로 줄이고 남은 블록

을 감사 가능하게 남기는 것이다. 검증된 세계의 구멍은 지울 수 없으니, 그 구멍을 좁히고 조명하는 일이 곧 보안 아키텍처가 된다.

관례는 규제가 아니다

Go는 동시성을 언어의 전면에 놓은 언어다. 고루틴은 만들기 싸서 수십에서 수천 개가 런타임에 공존하는 것이 흔한 현실이다. Uber가 이 현실의 규모를 실측했다. 4,600만 줄의 Go 코드와 2,100개 마이크로서비스를 6개월간 조사한 결과, 데이터 레이스 2,000건 이상을 발견하고 1,000건 이상을 수정했다. 수정에는 790개의 코드 패치와 210명의 개발자가 동원되었다.

문제의 성질이 중요하다. 채널과 뮤텍스의 올바른 사용은 Go가 강제하지 않는다. 공유 메모리로 통신하지 말라는 유명한 관용구는 문서 속의 권고일 뿐, 컴파일러가 집행하는 규칙이 아니다. 타입 시스템은 레이스에 대해 침묵한다. 그래서 검출은 런타임으로 밀려난다. Go 배포판에 내장된 레이스 검출기는 실행 관측에 의존하는 동적 검증이다. 실행해 본 적 없는 경로의 레이스는 잡지 못한다. 수정된 1,000건이 이미 실제로 겹쳐서 터졌던 사고라는 점이 그 뜻이다.

원본 논문의 규제 언어를 빌리면 이것은 사후 통제다. 미국 증권거래위원회의 시장 접근 규칙이 주문이 시장에 닿기 전에 걸러내는 사전 통제를 요구하는 것과 정반대 방향이다. 관례가 규제의 역할을 대신하려면 강제가 있어야 한다. 지금의 레이스 검출기는 강제가 아니라 사후 진단이고, 그 사이의 빈자리가 2,000건의 사고로 청구되었다.

계약이지만 선택적인 타입

Python은 점진적 타이핑이라는 세 번째 길을 택했다. 타입 주석을 쓸 수 있지만 쓰지 않아도 실행된다. 주석은 계약 문서의 형식을 갖췄는데, 그 계약을 검사하는 일은 별도의 도구를 별도로 돌려야만 일어난다. 검사기가 없는 프로젝트에서 주석은 희망에 가까운 기록이다. 계약이지만 강제가 없는 상태다.

RightTyper는 이 빈 계약서를 실측으로 채우는 도구다. 프로그램을 실제로 실행하며 관측된 값들의 타입을 수집하고, 이를 정적 분석과 결합해 타입 주석을 자동으로 생성한다. 관측에 의존하는 방식의 고전적 약점은 비용이다. 기존 동적 방식들은 최대 270배까지 느려졌다. RightTyper는 통계적으로 안내되는 적응형 샘플링으로 이 균형을 다시 잡아 오버헤드를 약 27퍼센트까지 낮췄고, 여러 비교 대상과 실제 코드에서 정답 및 개발자가 쓴 주석에 더 가까운 결과를 냈다.

파이썬이 서는 위치를 정리해 보자. 한쪽 끝은 확률적 관행이다. 주석의 유무와 품질이 프로젝트의 습관에 맡겨져 있다. 반대쪽 끝은 결정론적 검증이다. 검사기를 통과하지 못하면 진행이 막힌다. RightTyper는 관행의 세계에서 관측된 사실을 계약 문서로 끌어올리는 다리다. 다만 그 계약이 강제가 되려면 마지막 칸이 하나 더 필요하다. 검사기 통과를 병합의 조건으로 만드는 일이다.

마이크로초의 증명

언어 계층을 세 바퀴 돌았으니 원본 논문의 숫자 하나를 소환하자. 아마존웹서비스는 접근 제어 언어 Cedar를 Lean 4로 형식 검증하면서, 형식화된 모델로 입력 하나를 평가하는 데 평균 5마이크로초가 걸린다는 것을 실측했다. 잘 튜닝된 Rust 구현의 7마이크로초보다 빠르다. 형식 검증은 무겁다는 통념을 숫자가 뒤집는 지점이다.

두 계층의 비용과 보장을 나란히 놓으면 그림이 맞춰진다. 언어 계층의 가드레일은 컴파일마다 공짜로 집행된다. 대신 보장의 범위는 언어가 정의한 명제 안에서 끝난다. 소유권은 메모리를 지키지만 자원 한도를 지키지 않는다. 수학 계층은 정책 그 자체를 증명의 대상으로 만들어 보장을 사업의 규칙까지 연장하고, 그 비용마저 이제 마이크로초 단위로 내려왔다. 두 계층은 경쟁 관계가 아니라 계단이다.

세 언어 각각의 다음 계단도 같은 방향을 본다. 필자의 각색인 채로 말한다. Rust의 다음 계단은 unsafe 블록을 정리하고 남은 구멍을 감사로 조명하는 것이다. Go의 다음 계단은 레이스를 사후 진단이 아니라 정적 금지로 끌어올리는 것이다. Python의 다음 계단은 자동으로 채워진 계약에 강제를 붙여 검사기 통과 없이는 진행이 막히게 하는 것이다. 다음 표로 정리한다. 이 표도 필자의 각색이다.

언어	컴파일 시점에 강제	관례에 맡기는 것	런타임에 검사
Rust	소유권·수명·타입	unsafe 내부의 정확성	없음(사전 통제)
Go	기본 타입	채널·뮤텍스 사용 규범	레이스 검출기(동적)
Python	없음	타입 주석의 작성 관행	검사기 별도 실행(선택)
Lean 4(원본)	정책 공리의 컴파일	없음	커널의 증명 판정

강제와 관례의 경계가 곧 보안과 사고의 경계라는 사실은, 186건의 메모리 안전 버그와 2,000건의 데이터 레이스가 이미 지불한 수업료다. 원본 논문은 이 경계를 없애는 쪽으로 한 칸 더 간다. 모든 규칙을 증명의 대상으로 만들어, 관례라는 이름의 미결 상태를 남기지 않겠다는 것이다.

DAY

14

로드맵과 결론



1단계 — 그림자 속에서 번역을 검증하다

최소 실행 가능 제품(MVP) 단계의 목표는 생산 시스템을 위협에 빠뜨리지 않으면서 Aristotle의 자동 형식화 능력을 검증하는 것이다. 컴플라이언스 팀이 SEC 규칙 15c3-5에서 따온 기본 자본 한도, 기초 데이터 프라이버시 규칙 같은 정적이고 한정된 정책 부분집합을 Aristotle API에 입력한다. Aristotle이 이 자연어 지시를 Lean 4 공리로 번역한다. 형식화 드립트를 막기 위해 엔지니어링 팀은 MenTaL식 개념-심볼 제약을 깔아 금융 용어와 Lean 논리의 엄밀한 일대일 매핑을 강제한다. 시스템은 비동기 그림자 모드로 돈다. 살아 있는 에이전트의 출력을 기록해 두었다가 실행 뒤에 오케스트레이터가 검증하고, Lean 커널의 판정을 인간 컴플라이언스 리뷰와 대조해 Aristotle의 번역 충실도와 기준 오류율을 측정한다. 이 단계에서 아무것도 막지 않는다. 대신 무엇이든 막을 수 있게 되기 전에 번역의 품질이 먼저 계량된다. 관찰이 계량으로 이어지고, 계량이 신뢰로 이어지는 순서가 뒤집히지 않는다.

2단계 — 요격 게이트웨이로

2단계에서 프로토콜은 분석 도구에서 능동적인 동기 보안 게이트웨이로 이동한다. 검증을 마친 Lean 4 정책 환경을 강화된 WASM 런타임 안에 컴파일해 배포한다. WASI 역할을 엄격히 설정해 샌드박스가 호스트 네트워크와 파일시스템에 접근할 수 없게 만든다. 오케스트레이터가 에이전트 루프의 임계 경로로 들어간다. 실시간 API 요청을 가로채고, 에이전트의 제안 행동을 Lean 추측으로 만들고, SMT 기반 grind 전술로 서브밀리초 커널 검증을 해낸다. 여기에 '설명할 권리' 역번역 파이프라인이 붙는다. 커널이 반증한 행동은 Herald 데이터셋의 방법론을 통과해 평이한 언어의 불리조치 통지문으로 번역되고, ECOA와 FCRA의 요구를 충족한다. 1단계가 관찰이었다면 2단계는 통제다. 그리고 통제가 처음으로 실시간이 된다. 에이전트의 행동이 시장에 닿기 전에 검증이 끝난다.

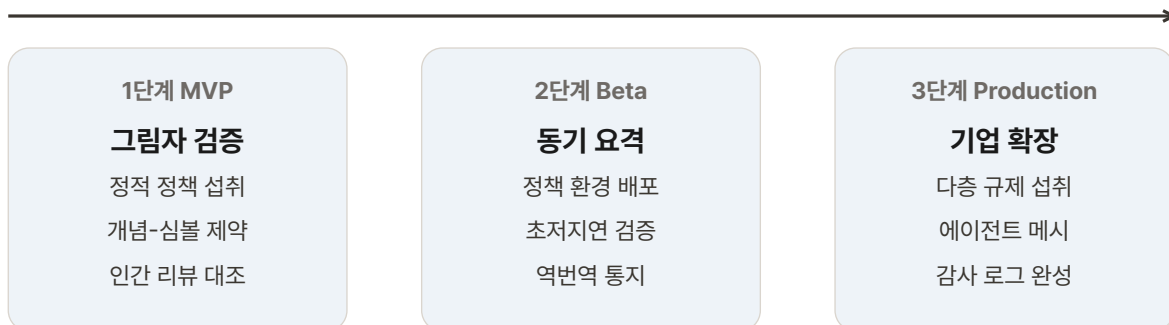
3단계 — 기업 규모의 증명 메시지

마지막 단계는 정책 환경을 글로벌 규제 전체로 확장한다. GDPR, EU AI Act, 바젤 III처럼 층을 이루는 복합 준수 규칙을 섭취한다. 오케스트레이션은 기계의 언어로 격이 올라간다. 경계가 정해진 MPSC(다중 생산자·단일 소비자) 사서함과 워크 스틸링 스케줄러 같은 고급 원시 기능이 동시 다발의 다중 에이전트 메시지를 떠받치고, 대규모 부하에서도 초저지연을 유지한다. 마무리는 감사다. 기관이 실행하는 모든 에이전트 행동이 실행 원격 데이터에 저장된 불변의 Lean 4 수학적 증명과 되돌릴 수 없이 연결된 암호학적으로 안전한 감사 로그로 완성된다. 기관은 이 지점에서 제로 트러스트이며 수학적으로 검증된 AI 인프라를 손에 넣는다. 확장이 부하만 늘리는 것이 아니라 증거도 함께 쌓는 구조다. 세 단계의 전개를 한 장으로 압축하면 아래와 같다.

ROADMAP

프로토콜은 세 위상으로 게이트를 강화한다

그림자 검증에서 실행 요격으로, 다시 확장으로 나아간다.



편집 요약: 본문의 3단계 로드맵을 위상으로 재배열한 도식이다.

규제와 수학의 화해

원문의 결론은 이 책 전체의 예고편이기도 했다. 해소하겠다고 선언한 문제는 확률적 에이전트 AI와 결정론적 규제 요구 사이의 근본 비양립이다. 해법은 규제를 소프트웨어의 바깥에 두지 않고 직접 매핑하는 것이었다. 시장 접속 규칙 15c3-5의 사전 차단 요건은 게이트웨이의 하드 가드레일에, OCC 고시 2011-12의 모델 리스크 관리에는 검증 가능한 모델 거버넌스에, FINRA 규칙 3110의 감독 체계에는 인간 통제 아래 100퍼센트 예측 가능한 감독 체계에, ECOA와 FCRA의 불리 조치 통지에는 역변역 파이프라인에, EU AI Act의 설명 가능성 요구에는 같은 파이프라인의 유럽판 응답에 각각 꽂힌다. 시장의 가드레일에 대한 판정도 내려졌다. NeMo Guardrails의 확률적 한계와 Guardrails AI의 논리적 얇음을 넘어선다. 그리고 AWS의 Cedar와 DeFi의 AMM 형식화가 이것이 사변이 아니라 이미 가동 중인 기술임을 증명한다. 원문이 그리는 최종 상태는 '규제 위반에 절대 면역인' 수학 검증 AI 인프라 위에서 에이전트 AI가 무한히 확장되는 세계다. 물론 이는 원문의 목표 선언이지 검증된 결과가 아니다.

한국 독자를 위한 총정리 — 필자의 각색

지금부터는 필자의 각색으로 10장부터 13장까지의 논의를 회수한다. 3단계 로드맵은 미국 금융기관의 이야기지만, 그 사다리는 우리의 지형에도 었을 수 있다. 열쇠는 자기 계층에서 원문 개념의 대응물을 찾는 것이다.

언어 계층부터 보자. 13장에서 본 Rust의 소유권 타입 시스템, Go의 관례에 맡겨진 동시성과 그 레이스 검출, Python의 타입 힌트는 이미 축소된 증명 게이트다. 시작점은 명확하다. 에이전트가 만들어 실행하는 코드에 타입 검사와 정적 분석 게이트를 의무로 붙이고, 그 게이트를 통과하지 못한 코드에는 실행 권한을 주지 않는 것이다. 프레임워크 계층이라면 12장의 Flutter, React Native, TypeScript가 준 가르침이 대답이다. 디자인 토큰처럼 정책을 이름 붙은 불변 값으로 코드 화하고, 화면과 기능이 그 값 안에서만 조립되게 하라. 플랫폼 계층의 독자에게는 10장과 11장의 공공·협업 시스템이 무대다. 전자정부프레임워크의 공통 컴포넌트나 CMS의 발행 승인 게이트처럼, 조직이 이미 운영하는 검증 관문에 '에이전트의 행동'이라는 새로운 통행자를 등록하는 일부터 시작할 수 있다. 관문은 이미 서 있으니 통행자의 자격 검사 규칙만 새로 쓰면 된다. 수학 계층, 다시 말해 프로토콜 그 자체를 다루고 싶은 독자에게는 원문의 순서를 그대로 권한다. 정책 몇 조항을 Lean 4로 옮기고, 그림자 모드에서 인간 리뷰와 대조하고, WASM 샌드박스에 었는다.

순서를 한데 묶으면 이렇다. 어느 계층이든 첫 주에 할 일은 문지기의 존재 확인이다. 우리 팀의 파이프라인에서 지금 어떤 검사가 실패를 막고 있는지 목록으로 쓰는 것. 다음으로 할 일은 권한의 정리다. 그 검사를 통과하지 못한 산출물이 실제로 어디까지 나가고 있는지 추적하는 것. 마지막은 경계의 고정이다. 통과한 것만 실행된다는 원칙을 예외 없이 코드로 박아 넣는 것. 어느 계층이든 방향은 하나다. 확률이 결정을 내리는 자리와 결정이 실행되는 자리를 분리하고, 후자에 검증 가능한 문지기를 세우는 것. 이 책이 보여 주려 한 것은 그 문지기가 이미 우리 손에 있다는 사실이다. 타입 검사기든, 정책 엔진이든, Lean 커널이든. 남은 일은 그 계단을 끝까지 오르는 것이다.